

프로님이 2시간 내로 과거 프로젝트 쇼케이스를 해 주신다

강사1분 + 멘토 형식

프로젝트 평가 반영이 결국에 된다 ->
나중에 나를 클라이언트에게 어필한다고 생각하고 성심성의껏 임하기.

1. 평가 기준 파악하기.

2. 프로님의 데이터로 해보기.
무슨 데이터일까? 아마 선박 관련 데이터가 아닐까..

3.

머신러닝 회귀

- AI

- > 인공지능

- 인공지능은 인간의 지능이 갖고 있는 기능을 갖춘 컴퓨터 시스템이며, 인간의 지능을 기계 등에 인공적으로 구현한 가장 큰 범주이다.

- > 머신러닝

- 기계가 학습할 수 있도록 하는 알고리즘과 기술을 개발하는 분야

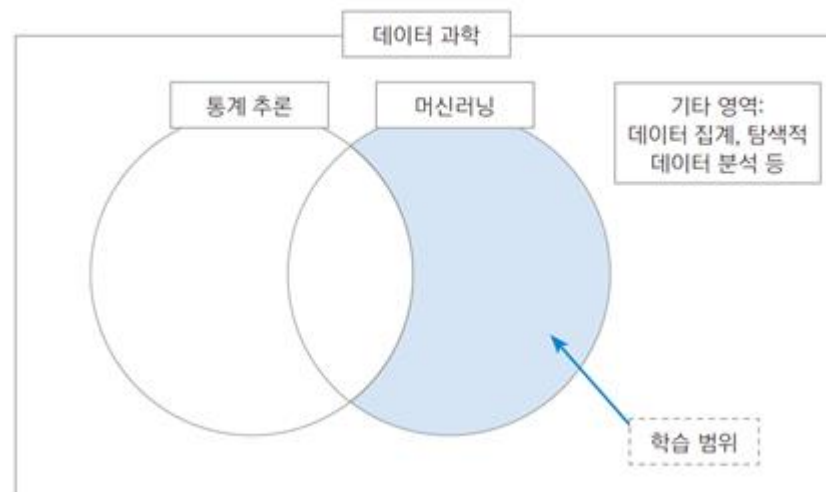
- > 딥러닝

- 인간의 뉴런구조와 비슷한 인공신경망 알고리즘을 사용하여 학습하는 분야



• 머신러닝이란?

- > 인과 관계(causal relation) 혹은 상관관계(correlation)를 모델링하는 기법을 사용하여 입력값(독립변수)과 출력값(종속변수)이 주어졌을 때의 알고리즘을 추정
- > 해결하려는 문제에 따라 예측(prediction), 분류(classification), 군집(clustering) 알고리즘 등으로 나뉜다.
- > 데이터 과학이라는 큰 영역에서 머신러닝은 통계 추론과 함께 큰 두 줄기를 구성함



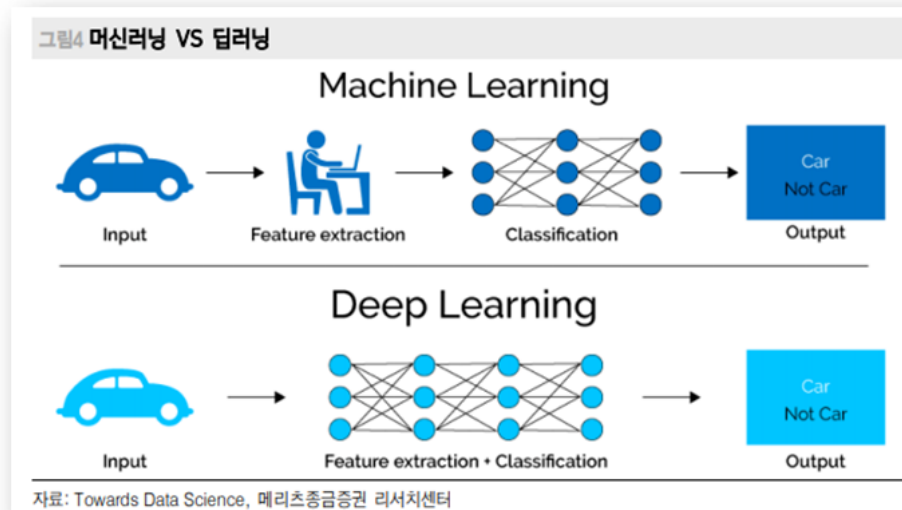
• 머신러닝 vs 딥러닝

> 머신러닝

- 데이터를 분석하고, 데이터로부터 학습한 다음, 학습한 것을 적용해 정보에 입각한 결정을 내리는 알고리즘

> 딥러닝

- 머신러닝의 하위 분야로 인공 신경망을 계층으로 구성하여 자체적으로 학습하여 결정을 내리는 알고리즘



머신러닝의 종류

> 머신러닝은 크게 3가지로 나뉜다.



• 머신러닝의 종류

> 지도 학습

- 사례들을 기반으로 예측을 수행한다.
- 입력 피쳐(feature)와 입력 피쳐에 해당하는 목표 변수의 쌍이 주어졌을 때 이 관계를 모델링하여 입력으로 들어온 새로운 피쳐로 목표 변수를 예측함.
- 분류, 회귀 등

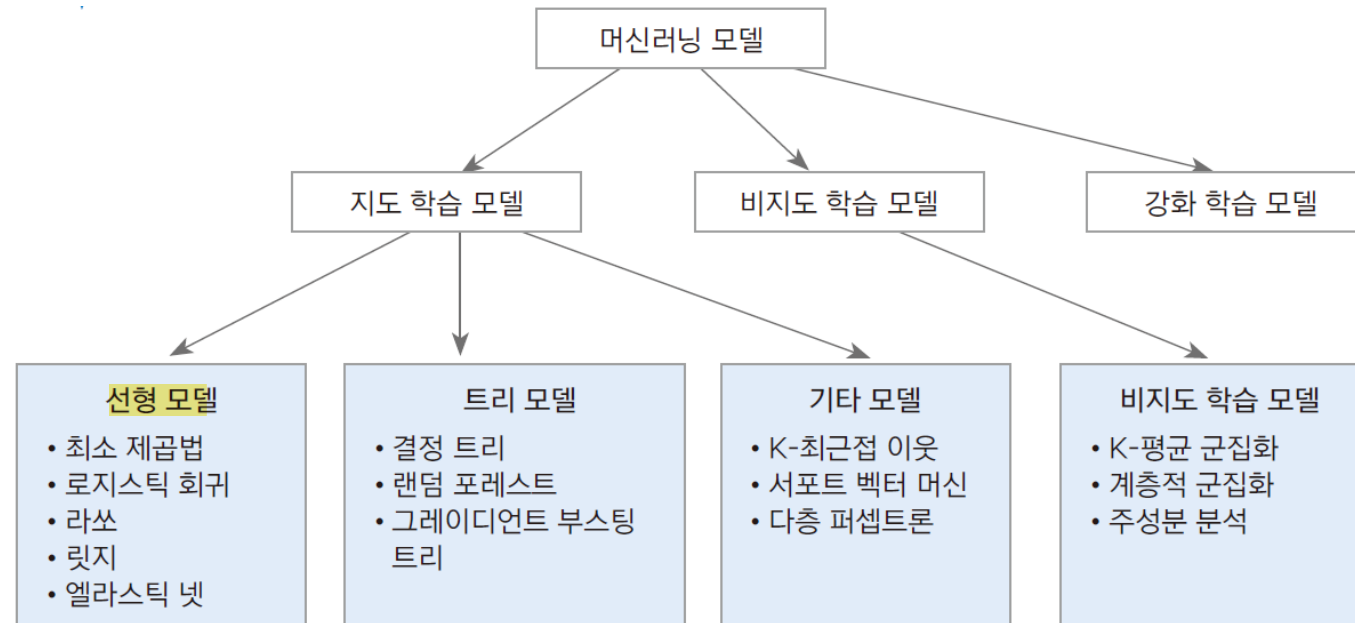
> 비지도 학습

- 데이터만을 학습하여 패턴을 찾아낸다.
- 피쳐 공간만 주어진 상태에서 이를 학습하여 피쳐 공간의 분포를 모델링하고 인사이트를 도출함.
- 군집화, 차원 축소, 이상값 탐지 등

> 강화학습

- 피드백을 기반으로 행동을 분석하고 최적화한다.
- 시행 착오와 지연 보상을 통해 최적의 방법을 스스로 찾아간다.

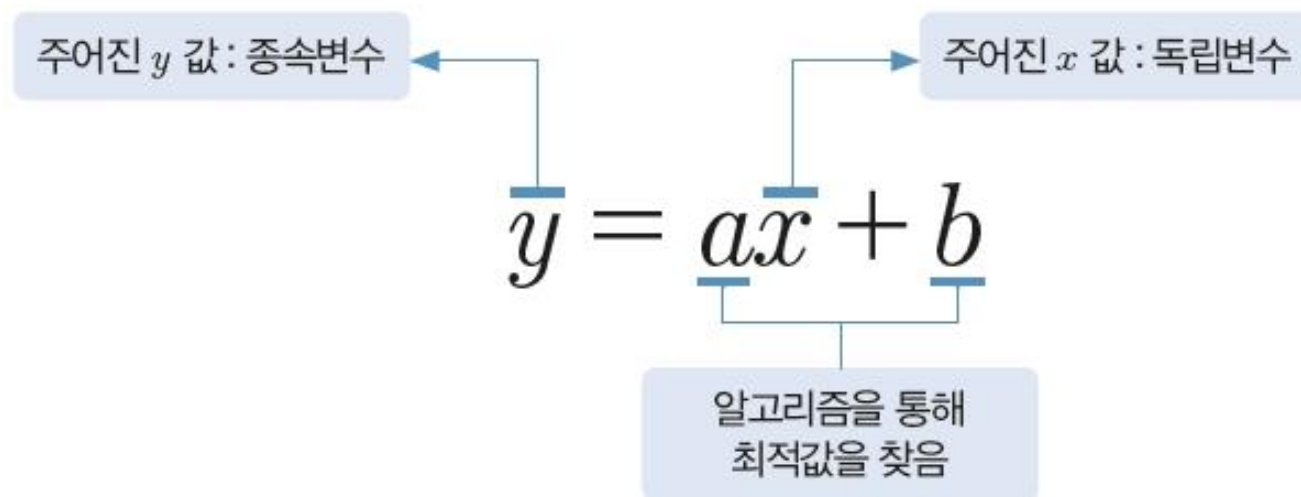
• 머신러닝의 알고리즘 종류



• 데이터의 이해

> 피쳐의 개념

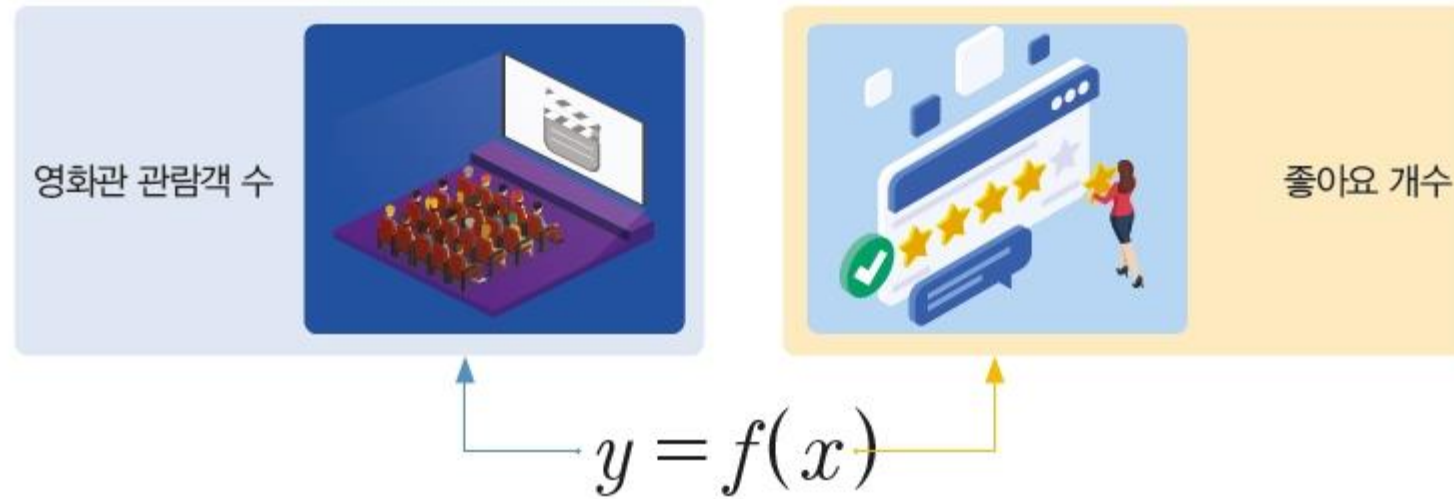
- 피쳐(feature) : 특성이나 특징이라는 의미
- 모델을 구성하는 데 데이터가 가장 큰 영향을 줌



• 데이터의 이해

> 피쳐의 개념

- 영화관 관람객 수(y)를 인터넷 웹사이트의 좋아요 개수(x)로 예측할 수 있다고 가정하면, 아래와 같은 간단한 식으로 표현할 수 있다.



• 데이터의 이해

> 보스턴 집값 예측(Boston House Price) 데이터 셋

x 변수 13개	[01] CRIM	자치시(town)별 1인당 범죄율
	[02] ZN	25,000 평방피트를 초과하는 거주지역의 비율
	[03] INDUS	비소매상업지역이 점유하고 있는 토지의 비율
	[04] CHAS	찰스강에 대한 더미변수(강의 경계에 위치한 경우는 1, 아니면 0)
	[05] NOX	10ppm 당 농축 일산화질소
	[06] RM	주택 1가구당 평균 방의 개수
	[07] AGE	1940년 이전에 건축된 소유주택의 비율
	[08] DIS	5개의 보스턴 직업센터까지의 접근성 지수
	[09] RAD	방사형 도로까지의 접근성 지수
	[10] TAX	10,000달러 당 재산세율
	[11] PTRATIO	자치시(town)별 학생/교사 비율
	[12] B	$1000(B_k - 0.63)^2$, 여기서 B_k 는 자치시별 흑인의 비율을 말함
	[13] LSTAT	모집단의 하위계층의 비율(%)
y 변수	[14] MEDV	본인 소유의 주택가격(중앙값) (단위 : \$1,000)

- 데이터의 이해

- > 보스턴 집값 예측(Boston House Price) 데이터 셋

- 범죄율, 방의 개수, 재산세율 등의 값(독립변수)들이 어떻게 집값(종속변수)에 영향을 주는지에 대한 모델
- 서로 다른 독립변수 13개를 x로 하고 가중치를 선형 결합하여 나타냄

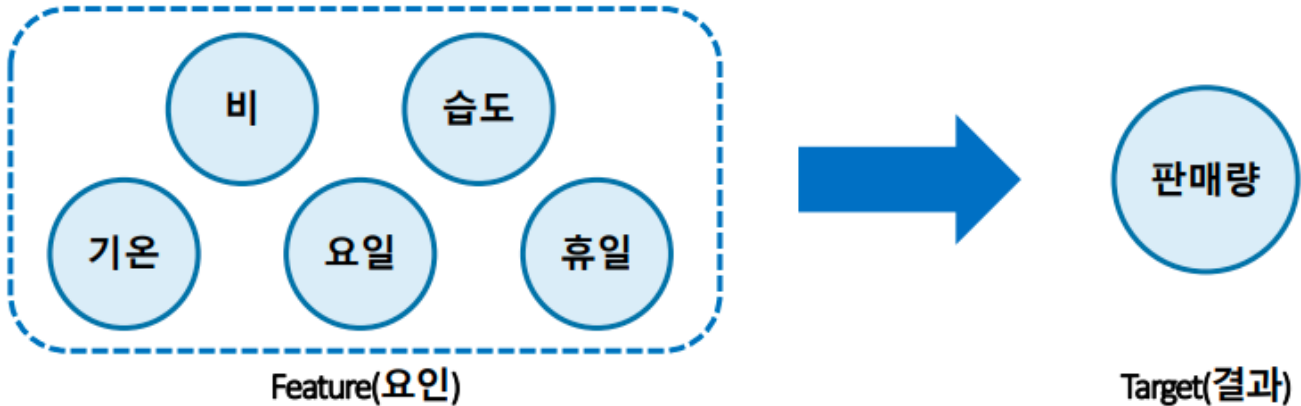
$$y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \dots + \beta_{13} x_{13} + \beta_0 \times 1$$

- > 머신러닝에서 독립변수 x를 피쳐라고 한다.

- 종속변수에 영향을 주는 특성, 데이터가 가지고 있는 특성

• 데이터 유형

> 분석 가능한 데이터



날짜	비	습도	기온	요일	휴일	판매량
2021.05.01	N	34	16	토	Y	4,560
2021.05.02	Y	50	26	일	Y	4,290
2021.05.03	N	42	12	월	N	2,340
...	...	...	...	...	...	...

• 데이터 유형

> 분석 가능한 데이터

① 변수

Target, y, Output,
Label, (종속변수)

Features, X, input, (독립변수)

Label	V01	V02	V03	V04	...	V##

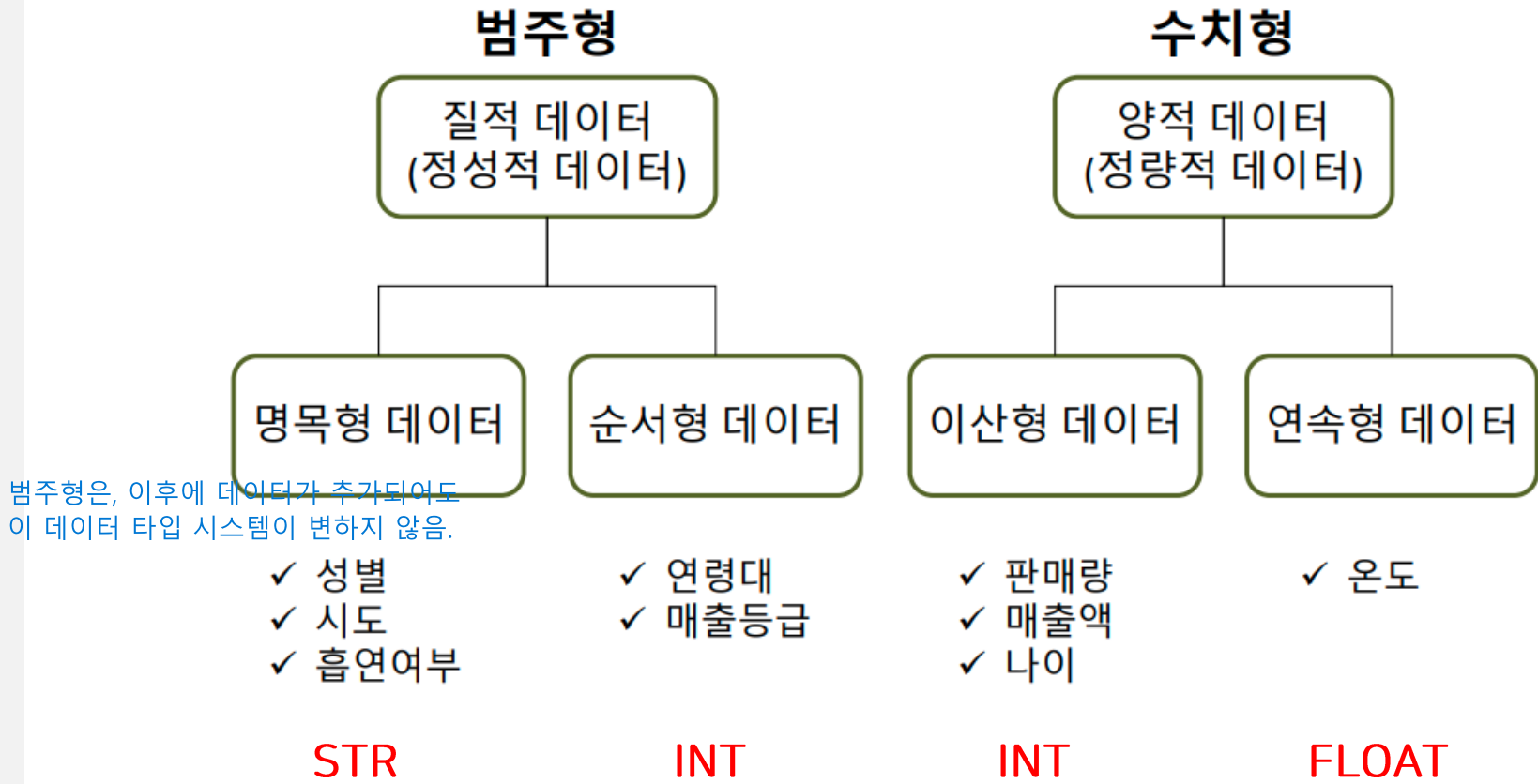
②
분석
단위

> 독립변수, 종속변수에 따라 분석 방법이 달라진다.

• 데이터 유형

데이터의 정해진 범위가 없음.

> 분석 가능한 데이터



• 빅데이터의 이해

> 빅데이터의 정의

- 기존 데이터베이스 관리 도구로는 수집, 저장, 처리 및 분석하기 어려울 정도로 방대하고 복잡한 대규모 데이터

반정형 데이터
데이터 '들'은 있지만 값은 정해져 있지 않음

html,dictionary,json

key,value 들은 주어져지만,
실제 값은 사용자가 정의함.

데이터 형식에 따라
저장하는 방식이 달라진다.

- 단순한 '데이터의 크기'를 넘어, 이를 분석해 가치 있는 인사이트를 도출하고 미래를 예측하는 기술과 과정 전체를 의미

> 빅데이터의 주요 특징

구분	영어	핵심 설명
기본 3V (투자 관점)	Volume(규모)	데이터의 양이 방대함 (TB, PB, EB 단위)
	Variety(다양성)	정형 + 비정형 등 데이터 형태가 다양함 (가장 중요한 특징)
	Velocity(속도)	데이터 생성 및 처리 속도가 매우 빠름 (예 : 시간 스트리밍) 하드웨어의 속도도 중요하다.
확장 +2V	Veracity(신뢰성)	데이터의 품질과 정확성 (믿을 수 있는가?) 신뢰할 수 있는 데이터를 사용해야 한다.
	Value(가치)	비즈니스에 유의미한 가치를 주는가? 이 데이터로 뭘 할 수 있는가?

시리: 음성 -> 텍스트 변환

내가 원하는 데이터를 쉽게 선택할 수 있다.
(정형)
+ 여러 형식의 데이터를 다룰 수 있다.
(반정형)

• 빅데이터의 이해

> 출현 배경과 변화

컴퓨터와 인터넷은
2차 세계대전 과정에서 개발되었다.

컴퓨터: 암호해독
인터넷: 군사 통신.

- 빅데이터 현상은 없었던 것이 새로 등장한 것이 아니라 기존의 데이터, 처리방식, 다루는 사람과 조직 차원에서 일어나는 변화를 가르킴
- 데이터 축적 및 활용 증가, 인터넷 확산, 저장 기술의 발전과 가격 하락, 모바일 시대의 도래와 스마트 단말의 보급, 클라우드 컴퓨팅 기술 발전, SNS와 사물 네트워크 확산 등 원인으로 발전



데이터 규모	EB(엑사바이트) (1990년대 말=100EB)	ZB(제타바이트) 진입 (2011년대 말=1.8ZB)	ZB 본격화 시대 (2020년=2011년 대비 50배 증가)
데이터 유형	정형 데이터 (데이터베이스, 사무 정보)	비정형 데이터 (문, 멀티미디어)	사물 정보, 인지 정보 (RFID, 센서, 사물 정보)
데이터 특성	구조화	다양성, 복합성, 소셜	현실성, 실시간성

• 빅데이터의 이해

> 빅데이터가 만들어 내는 본질적인 변화

- 통계 분석 → 빅데이터 분석
- 사전처리 → 사후처리
- 표본조사 → 전수조사
- 질 → 양
- 인과관계 → 상관관계

좋은 통계 모델은 데이터를 적게 사용한다.
왜냐면 좋은 표본을 기준으로 모델을 개발하니까.

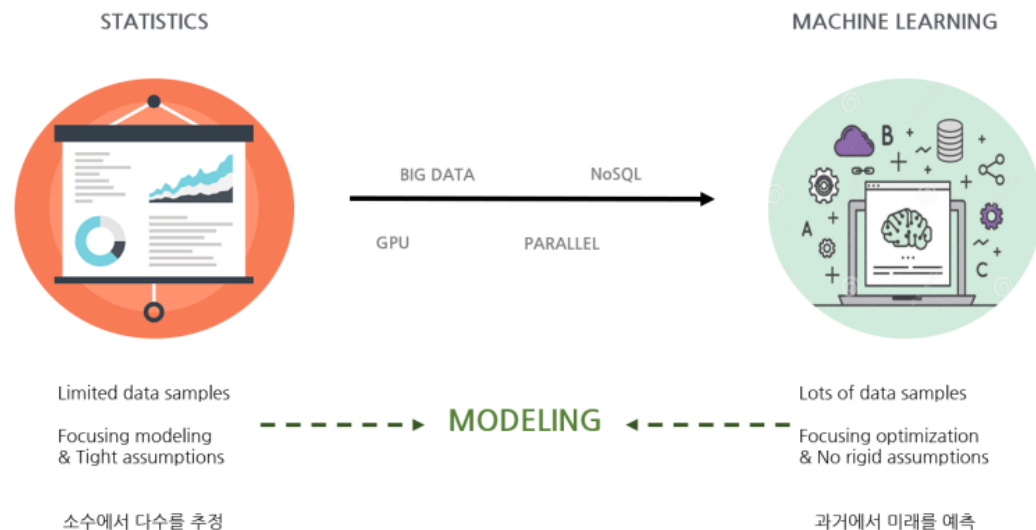
좋은 결과가 나오려면 사전처리가 필수였다.
하지만 데이터가 많아지면서, 오류값을 제거해 나가는 사후처리가 많아짐.

이제 전수조사를 해도 될 만큼 gpu성능이 좋아졌다.

이제는 데이터의 품질보다는, 신뢰성 있는 대량의 데이터가 필요.

통계에서는 원인 결과 과정이 중요하다.
상관관계:

Not only SQL?
JSON등을 저장하는 db



• 비즈니스 모델

음.. 머신러닝이랑, 데이터분석 이런 것들을 복합적으로 활용할 수 있는 '회귀분석'으로 할 수 있는게 뭘까?

뭔가 사용자가 변수값을 직접 조정해서 시뮬레이션 할 수 있는 거면 좋겠다.

> 빅데이터를 활용한 기본 테크닉

테크닉	내용	예시
연관규칙학습	변인들 간에 주목할 만한 상관관계가 있는지 찾아내는 방법	커피를 구매하는 사람이 탄산음료를 더 많이 사는가?
유형분석	문서를 분류하거나 조직을 그룹으로 나눌 때, 또는 온라인 수강생들을 특성에 따라 분류할 때 사용	이 사용자는 어떤 특성을 가진 집단에 속하는가?
유전자 알고리즘	최적화가 필요한 문제의 해결책을 자연 선택, 돌연변이 등과 같은 매커니즘을 통해 점진적으로 진화 시켜 나가는 방법	최대한 시청률을 얻으려면 어떤 프로그램을 어떤 시간대에 방송해야 하는가?
기계학습	훈련 데이터로부터 학습한 알려진 특성을 활용해 예측하는 방법	기존의 시청 기록을 바탕으로 시청자가 현재 보유한 영화 중에서 어떤 것을 가장 보고 싶어할까?
회귀분석	독립변수를 조작함에 따라, 종속변수가 어떻게 변하는지를 보면서 두 변인의 관계를 파악할 때 사용	구매자의 나이가 구매 차량의 타입에 어떤 영향을 미치는가?
감정분석	특정 주제에 대해 말하거나 글을 쓴 사람의 감정을 분석	새로운 환불 정책에 대해 고객의 평가는 어떠한가?
소셜네트워크분석 (=사회관계망분석)	특정인과 다른 사람이 몇 촌 정도의 관계인가를 파악할 때 사용하고, 영향력있는 사람을 찾아낼 때 사용	고객들 간 관계망은 어떻게 구성되어 있나?

추천 알고리즘이 따로 있다고 한다..

• 비즈니스 모델

> 빅데이터 분석 기술

- 하둡

하드디스크와 메모리 공유인듯?
큰 회사에서 많이 사용한다.

- 여러 개의 컴퓨터를 하나인 것처럼 묶어 대용량 데이터를 처리하는 플랫폼 기술
- 맵 리듀스로 분산파일 시스템에 저장된 대용량 데이터를 대상으로 SQL을 이용해 사용자 질의를 실시간으로 처리

- Apache Spark =pandas

- 실시간 분산형 컴퓨팅 플랫폼
- 스칼라로 작성되어 있지만 스칼라, 자바, R, 파이썬, API 지원
- In-Memory 방식으로 처리 → 하둡보다 처리속도가 빠름

- Smart Factory

- 공장 내 설비와 기계에 IoT 설치로 생산성 극대화

- Machine Learning & Deep Learning

- 인공지능 연구 분야 중 하나
- 인간의 학습 능력과 같은 기능을 컴퓨터에서 실현하고자 하는 기법

• 데이터 분석 방법

20

→ > EDA(Exploratory Data Analysis) 탐색

- 여러 변수 간 트렌드, 패턴, 관계를 찾는 것으로, 통계적 모델링이 아닌 다양한 관점에서 데이터를 분석하는 방법
- 데이터의 통계량, 그래프 등으로 파악 info로 찍어보면서 데이터를 파악하는 단계
- 인사이트나 가설을 세우는 단계 데이터간 연관관계를 탐색하고, 가설을 세우는 과정.

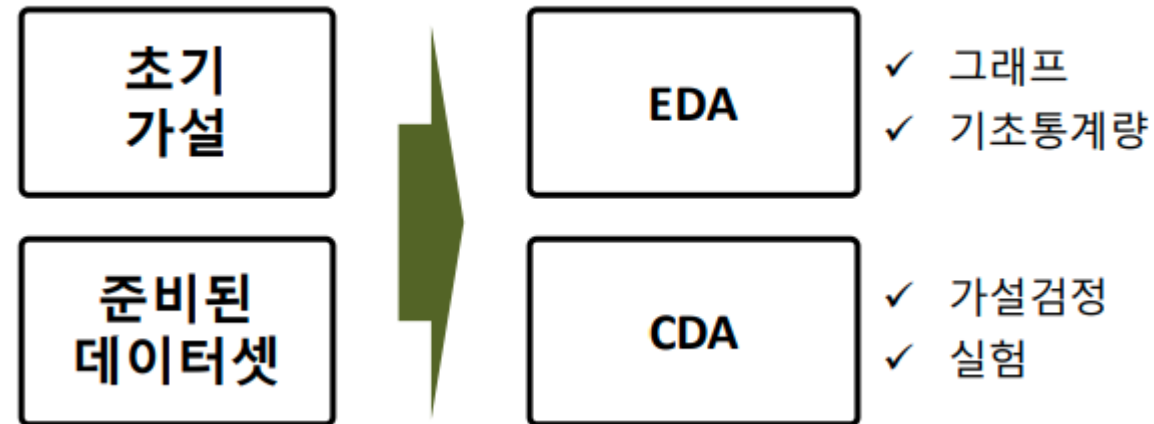
> CDA(Confirmatory Data Analysis) 확증

- 탐색으로 파악하기 어려운 부분을 통계적 분석 도구를 통해 분석하는 방법
- 통계적 추론과 가설검정을 통한 증명
- EDA에서 찾은 인사이트나 가설을 검증하는 단계

• 데이터 분석 방법

> EDA & CDA

- 가설 & 데이터를 사용하여 데이터 분석을 진행하는 방법이다.



> 데이터를 확인하여 다음과 같은 방식으로 목표를 설정해야한다.

- 1. (언제, 어떤) 그래프를 그리고 (어떻게) 해석할지
- 2. (언제, 어떤) 기초통계량을 구하고 (어떻게) 해석할지
- 3. (언제, 어떤) 가설검정 방법을 사용하고 (어떻게) 해석할지

- 단변량 분석

우리가 관심있는 데이터 = Survived

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

```
import pandas as pd
```

```
titanic = pd.read_csv('data/titanic.csv')
```

```
titanic
```

Out :

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

• 단변량 분석

> 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

	기초 통계량 (숫자로 요약하기)	시각화	
숫자형 양적 데이터 (정량적 데이터)	min, max, mean, std 사분위수	Histogram Density plot Box plot	조각조각 표현하기 쉬운 애들.
범주형 질적 데이터 (정성적 데이터)	범주별 빈도수 범주별 비율	Bar plot Pie chart	범위,비율로 표현해야하는 애들

범주형은 빈도수,비율 등의 숫자로 바꿔야하네.

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

```
titanic['Age'].describe()
```

```
Out :      count      714.000000  
      mean      29.699118  
      std       14.526497  
      min        0.420000  
      25%       20.125000  
      50%       28.000000  
      75%       38.000000  
      max       80.000000  
      Name: Age, dtype: float64
```

- > 해석

- 가장 어린 나이는 0.42세, 최고령자는 80세입니다.
- 탑승객의 평균은 29.7세이다.
- 탑승객의 75%는 38세 이하로 많은 사람이 젊은 층이다.

- 단변량 분석

> 숫자형, 범주형 데이터

```
import matplotlib.pyplot as plt
import seaborn as sns

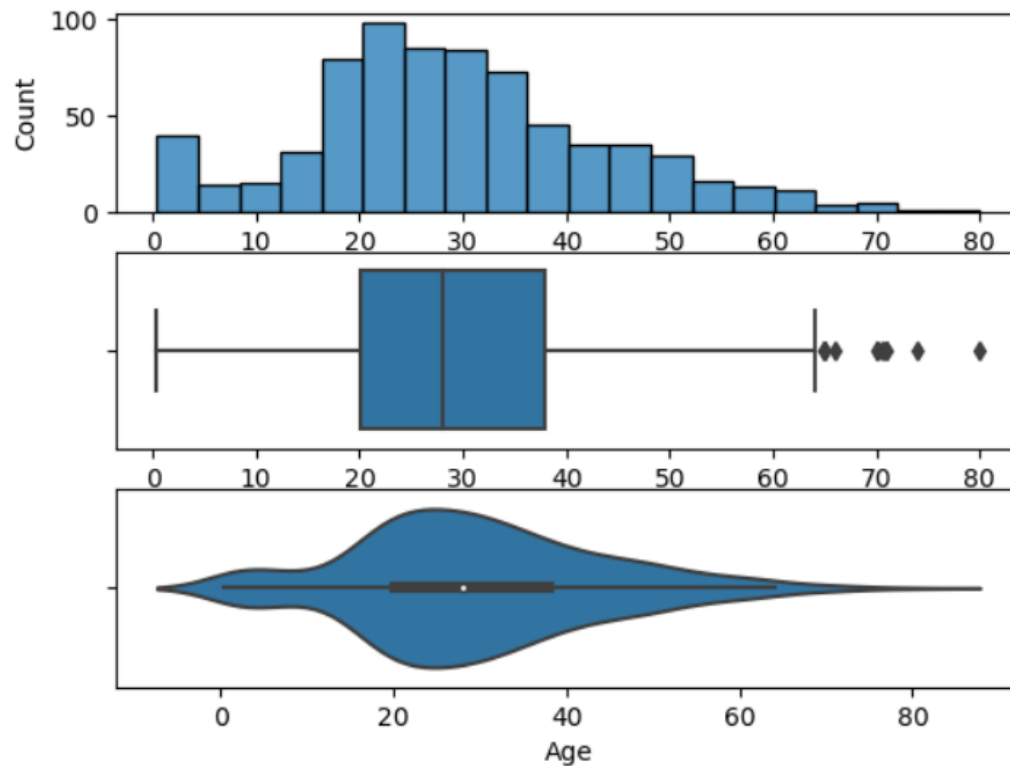
plt.subplot(3,1,1)
sns.histplot(data = titanic, x = 'Age')
plt.subplot(3,1,2)
sns.boxplot(data = titanic, x = 'Age')
plt.subplot(3,1,3)
sns.violinplot(data = titanic, x = 'Age')
```

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

Out :



- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

```
titanic['Fare'].describe()
```

Out :

count	891.000000
mean	32.204208
std	49.693429
min	0.000000
25%	7.910400
50%	14.454200
75%	31.000000
max	512.329200
Name: Fare, dtype: float64	

- > 해석

- 요금의 범위는 \$0 ~ \$512로, \$0가 있다는 것은 요금을 내지않는 나이대가 있음을 알 수 있다.
- 평균 요금은 \$32.2이지만 75%가 \$31 이내인 것을 보아 데이터는 왼쪽으로 굉장히 치우쳤음을 알 수 있다.

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

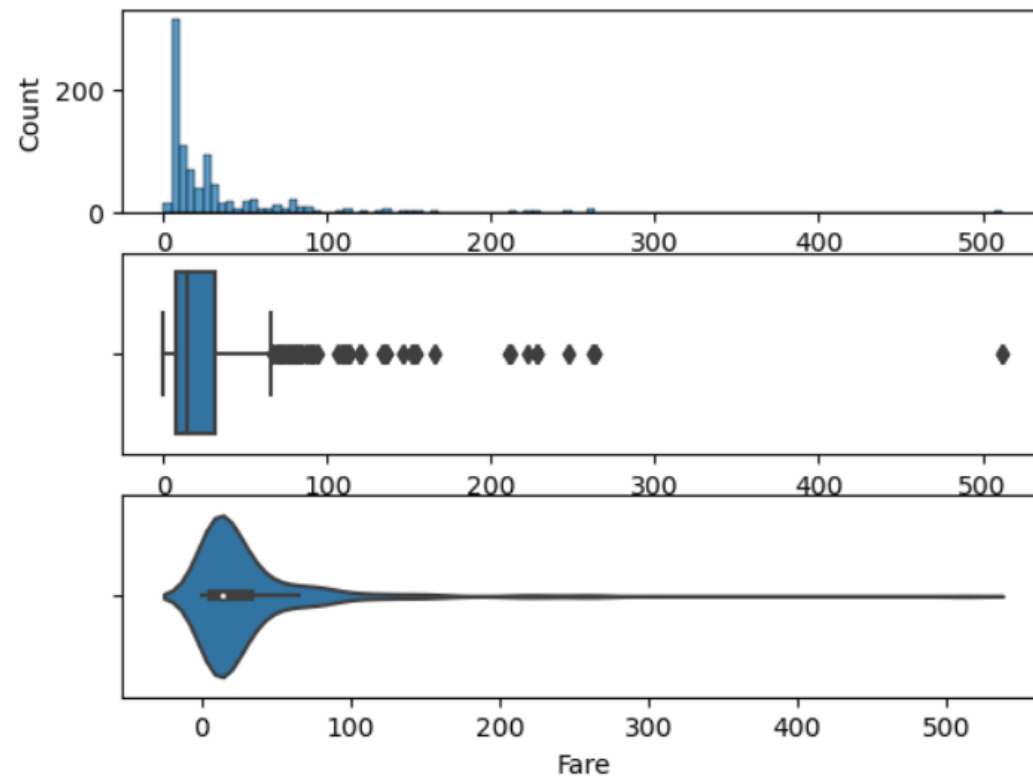
```
plt.subplot(3,1,1)
sns.histplot(data = titanic, x = 'Fare')
plt.subplot(3,1,2)
sns.boxplot(data = titanic, x = 'Fare')
plt.subplot(3,1,3)
sns.violinplot(data = titanic, x = 'Fare')
```

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

Out :



• 단변량 분석

> 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

이상치 판단

IQR

```
IQR = titanic['Fare'].quantile(0.75) - titanic['Fare'].quantile(0.25)
```

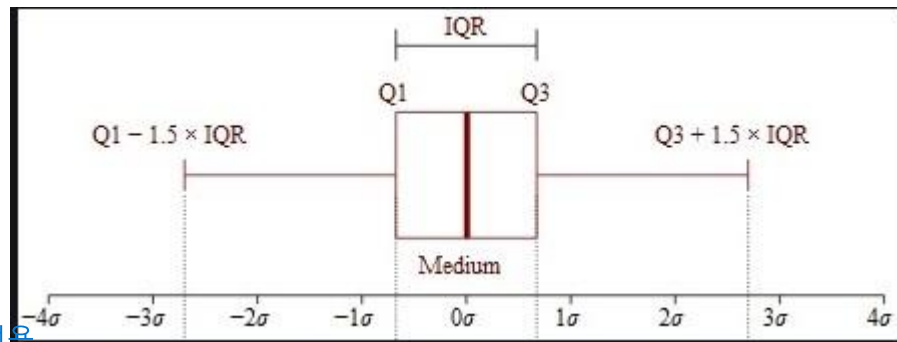
```
high_border = titanic['Fare'].quantile(0.75) + IQR * 1.5
```

```
low_border = titanic['Fare'].quantile(0.25) - IQR * 1.5
```

```
print(high_border, low_border)
```

plot과 axis는 별개로 이해하기.

Out : 65.6344 -26.724



아래쪽 x axis는 4분위수 전용.

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

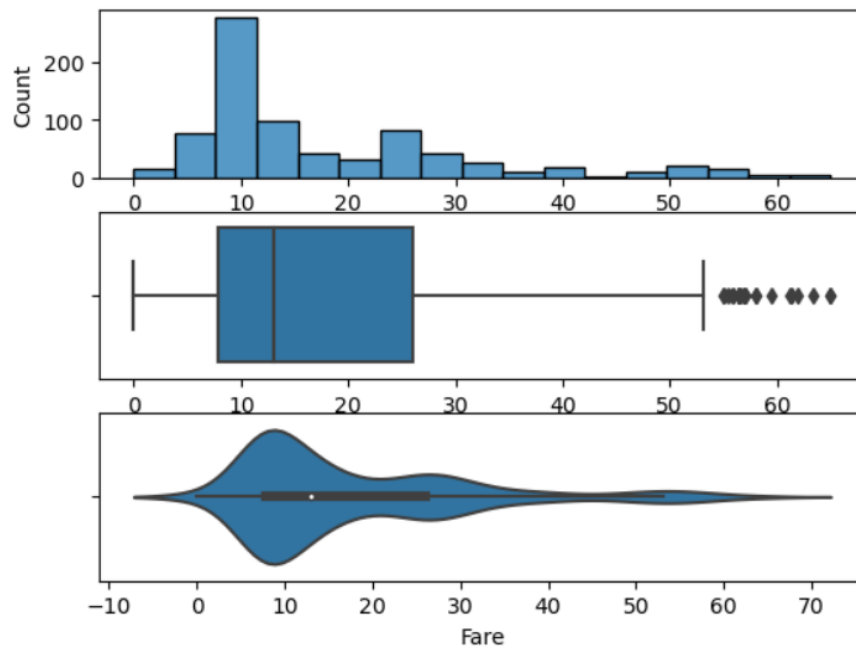
```
titanic2 = titanic[(titanic['Fare'] < high_border) & (titanic['Fare'] > low_border)]  
plt.subplot(3,1,1)  
sns.histplot(data = titanic2, x = 'Fare')  
plt.subplot(3,1,2)  
sns.boxplot(data = titanic2, x = 'Fare')  
plt.subplot(3,1,3)  
sns.violinplot(data = titanic2, x = 'Fare')
```

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

Out :



• 단변량 분석

> 숫자형, 범주형 데이터

z value로 계산하기.

- 통계량 또는 그래프를 통한 EDA

이상치 판단

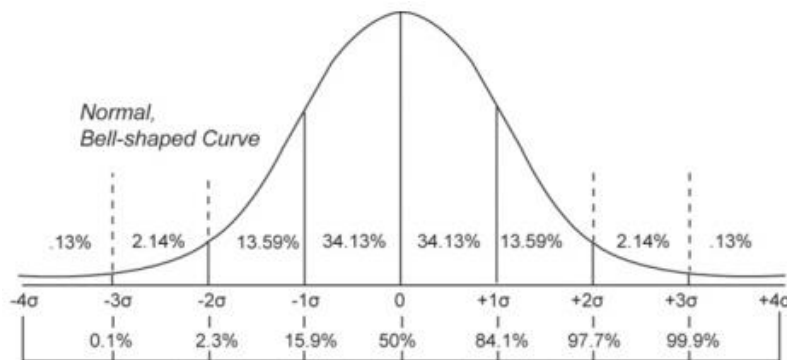
zscore : 평균에서 떨어진 정도로 이상치 검출, 2~3의 값으로 검출

```
import numpy as np
```

```
z_score = np.abs(titanic['Fare'] - titanic['Fare'].mean()) / titanic['Fare'].std()
```

```
titanic3 = titanic[z_score < 2.5]
```

$$Z = \frac{(X - \text{평균})}{\text{표준편차}}$$



- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

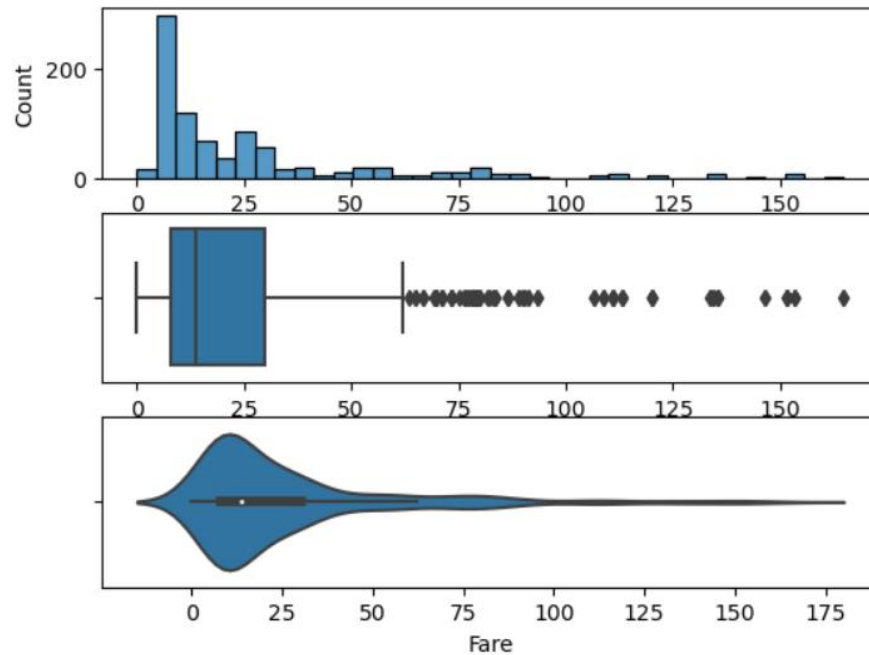
```
plt.subplot(3,1,1)
sns.histplot(data = titanic3, x = 'Fare')
plt.subplot(3,1,2)
sns.boxplot(data = titanic3, x = 'Fare')
plt.subplot(3,1,3)
sns.violinplot(data = titanic3, x = 'Fare')
```

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

Out :



65보다 큰 값 삭제 이후

분류: 이상치가 크게 중요하지 않음.

분석(예측): 이상치가 없어야함.

- 단변량 분석

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

```
print(titanic['Pclass'].value_counts())  
print(titanic['Pclass'].value_counts() / len(titanic['Pclass']))
```

```
Out :    3    491  
      1    216  
      2    184  
      Name: Pclass, dtype: int64  
      3    0.551066  
      1    0.242424  
      2    0.206510  
      Name: Pclass, dtype: float64
```

- > 해석

- 전체 비율 중 3등급이 55%를 차지한다.
- 1등급은 24%, 2등급은 20%로 가장 적다.

• 단변량 분석

> 숫자형, 범주형 데이터 =개수세기

- 통계량 또는 그래프를 통한 EDA

```
p_cnt = titanic['Pclass'].value_counts()

plt.figure(figsize = (7, 3))
plt.subplot(1,2,1)
sns.countplot(data = titanic, x = 'Pclass')
plt.subplot(1,2,2)
plt.pie(p_cnt.values, labels = p_cnt.index, autopct = '%.2f%%')
```

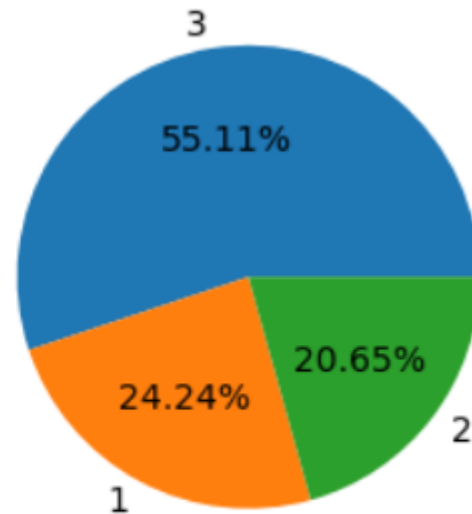
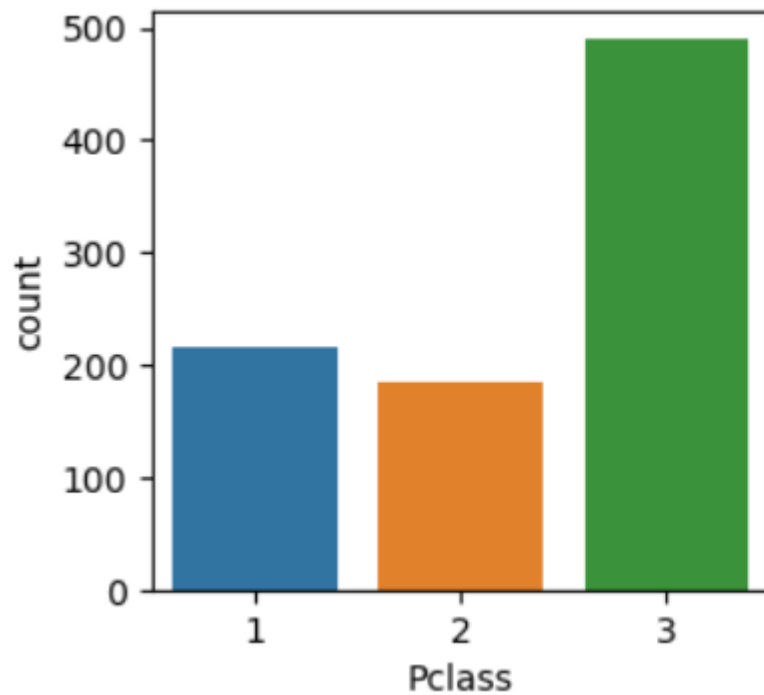
- 단변량 분석

1개의 변수를 분석한다.

- > 숫자형, 범주형 데이터

- 통계량 또는 그래프를 통한 EDA

Out :



• 이변량 분석

변수 2개 이상.

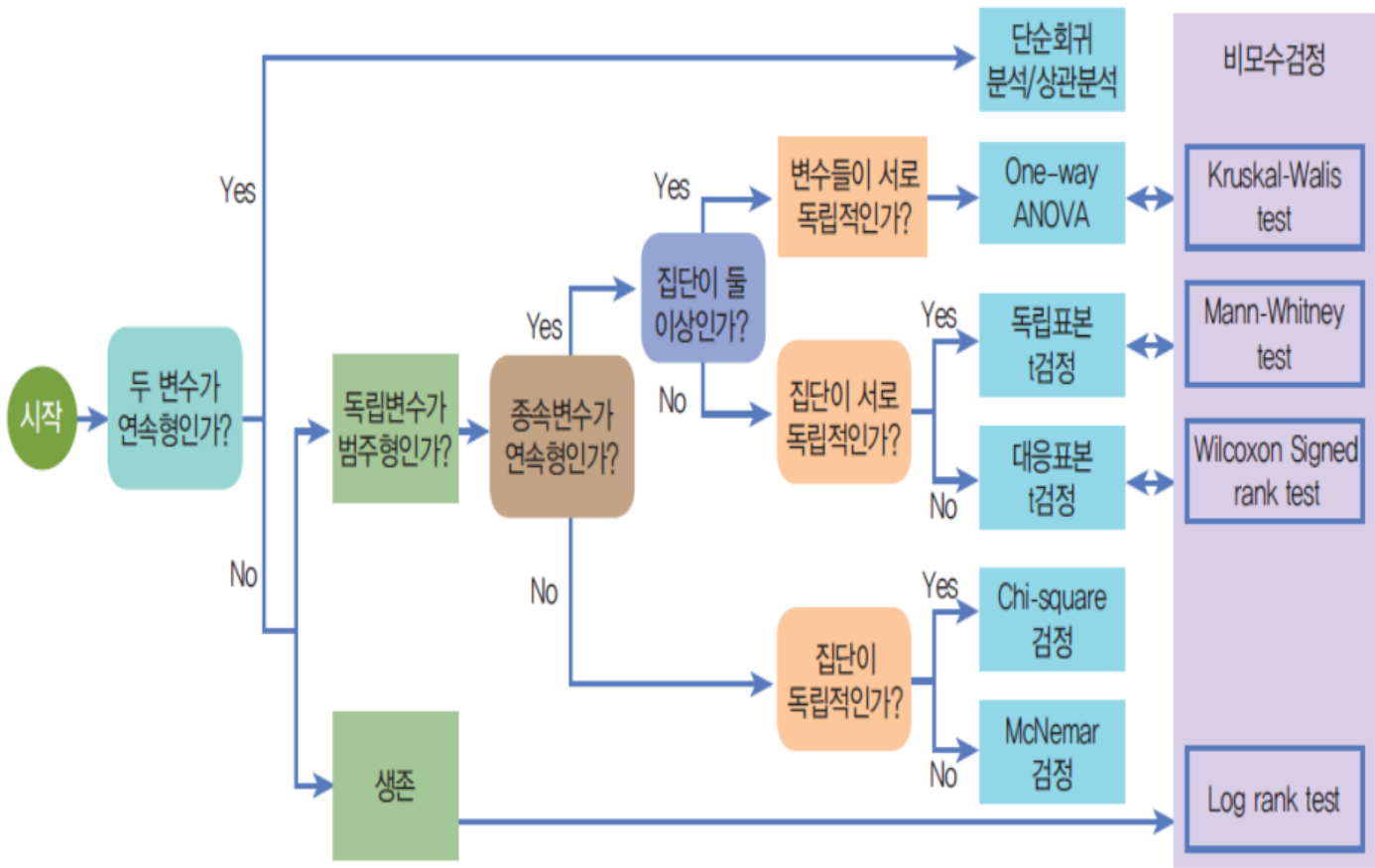
가설: A->B가 영향이 있을까?
예) 성별이 생존여부에 차이가 있을까?

가설검정

> 데이터 유형에 따른 분석 방법

A와 B의 데이터 유형에 따라 검정방법을 결정한다.

모수검정: 모집단이 정규분포를 따른다고 가정
비모수검정: 정규분포를 따르지 않는 경우



• 이변량 분석

> 데이터 유형에 따른 분석 방법

> 모수적 방법(정규성O)

- 두 기간에서의 변화를 분석할 때에는 대응표본 t검정(Paired T test)
- 하나의 기간에서 서로 다른 2개의 집단 : 독립표본 t검정(Independent samples T test)
- 하나의 기간에서 3개 이상의 집단 : 일원배치 분산분석(One-way ANOVA)
- 하나의 집단에서 3개 이상의 집단(등분산X) : 웰치스 분산분석(Welch's ANOVA)

> 비모수적 방법(정규성X)

- 집단이 하나이고 두 기간에서의 변화 : 윌콕슨 부호순위 검정(Wilcoxon signed rank test)
- 하나의 기간에서 서로 다른 2개의 집단 : 맨휘트니 검정(Mann-Whitneytest)
- 하나의 기간에서 3개 이상의 집단 : 크루스칼-월리스검정(Kruskal-Walis test)
- 하나의 집단에서 3개 이상의 기간에서의 변화 : 후리드만 검정(Friedman test)

• 이변량 분석

> 데이터 유형에 따른 분석 방법

결과 : y, target, label, 종속변수

원인 : X,
feature,
독립변수

		연속형(숫자)		범주형	
원인 : X, feature, 독립변수	연속형 (숫자)	Scatter (산점도)	상관 분석	Histogram Densityplot	로지스틱 회귀
	범주형	평균비교 Barplot	T 검정 F 검정	Countplot	카이제곱 검정

- 이변량 분석

- > 숫자 vs 숫자

- 상관분석 또는 산점도를 이용하여 분석

```
air = pd.read_csv('data/ozone.csv')  
air.head()
```

Out :

	Ozone	Solar.R	Wind	Temp	Date
0	41	190.0	7.4	67	1973-05-01
1	36	118.0	8.0	72	1973-05-02
2	12	149.0	12.6	74	1973-05-03
3	18	313.0	11.5	62	1973-05-04
4	19	NaN	14.3	56	1973-05-05

- > 온도(숫자)에 따른 오존 농도(숫자)의 관계성 분석

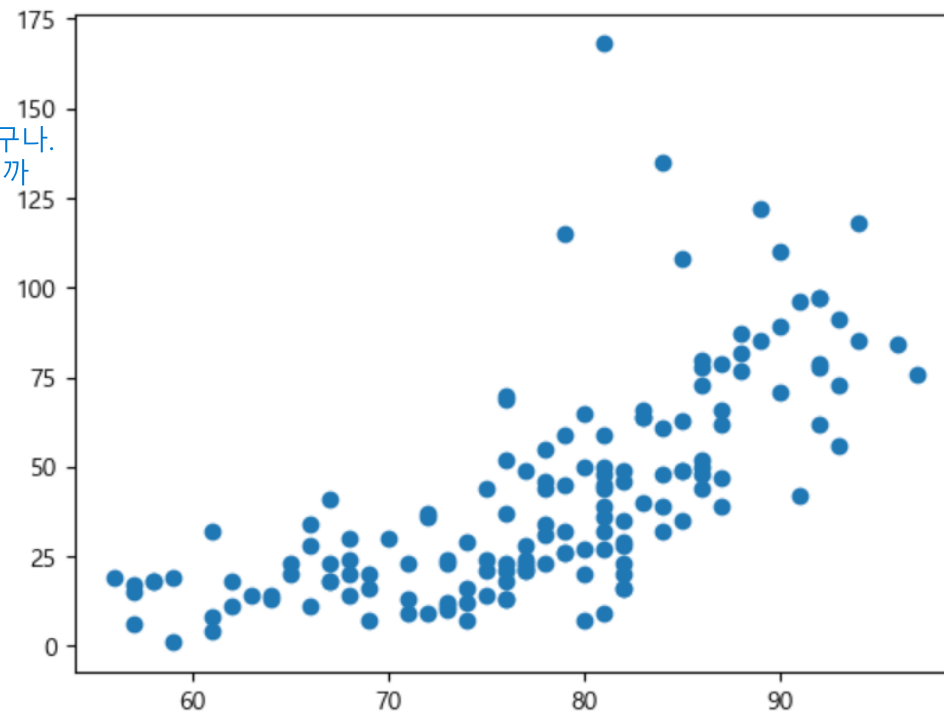
- 이변량 분석

- > 숫자 vs 숫자

- 산점도를 이용하여 분석

```
plt.scatter('Temp', 'Ozone', data=air)
```

Out :



온도와 오존은 어느정도 상관관계가 있구나.
-> 높이가 높아지면, 태양과 가까워지니까
오존량도 늘어나겠구나.

끝

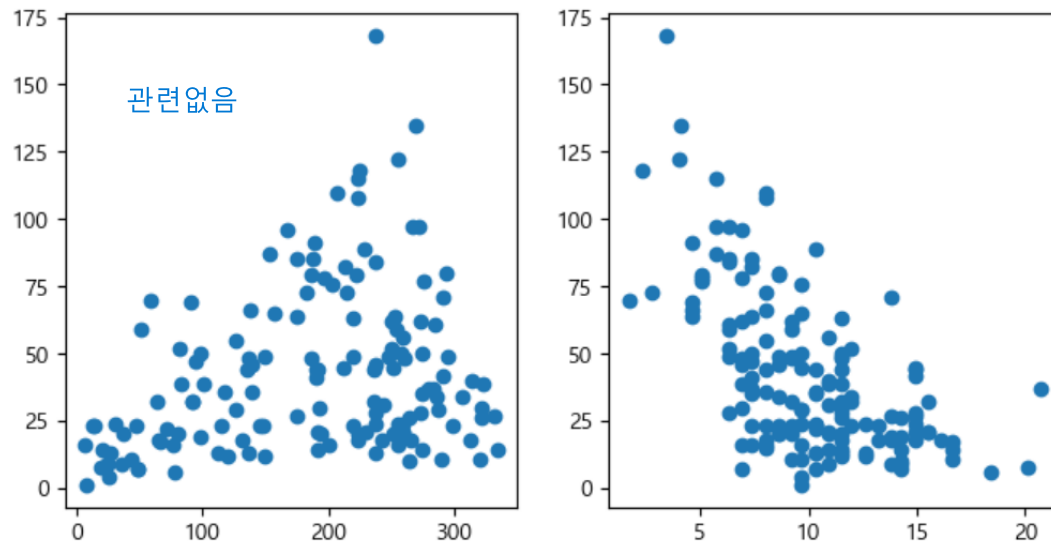
- 이변량 분석

- > 숫자 vs 숫자

- 산점도를 이용하여 분석

```
plt.subplot(1,2,1)  
plt.scatter('Solar.R', 'Ozone', data=air)  
plt.subplot(1,2,2)  
plt.scatter('Wind', 'Ozone', data=air)
```

Out :



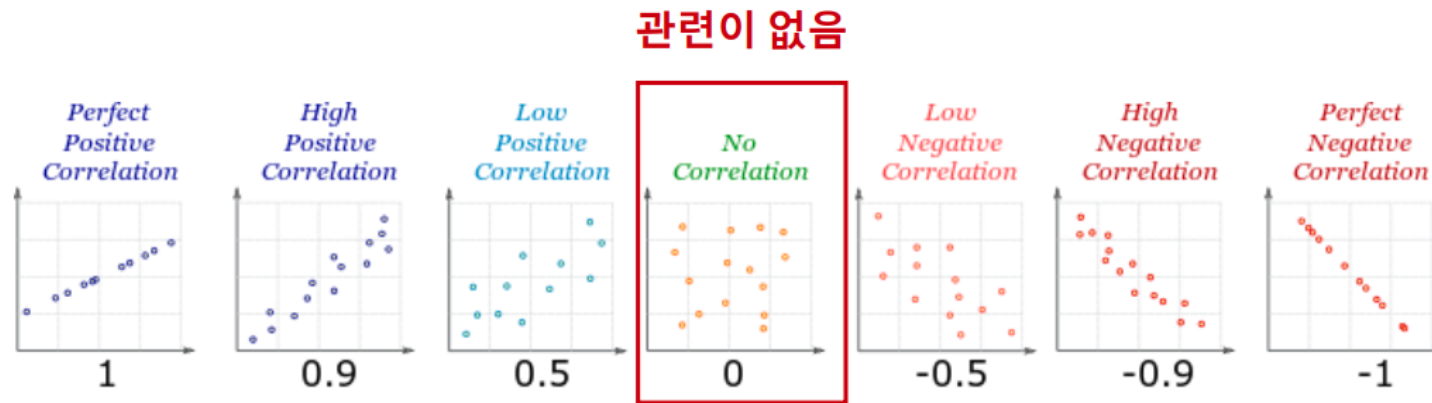
• 이변량 분석

비례/반비례 관계 -> linear한 correlation

그외: no correlation

> 숫자 vs 숫자

- 산점도를 통한 두 변수의 관계
- 뚜렷한 직선이 보인다면 강한 관계로 볼 수 있다.



- 이변량 분석

- > 숫자 vs 숫자

- 상관 계수를 통한 분석

- > 상관 계수란

- 데이터가 얼마나 직선으로 모여 있는지를 수치화한 값

- > 상관계수 공식

$$r = \frac{\frac{\sum (x - \bar{x})(y - \bar{y})}{n - 1}}{\sqrt{\frac{\sum (x - \bar{x})^2}{n - 1} \times \frac{\sum (y - \bar{y})^2}{n - 1}}}$$

- -1 ~ 1의 값을 가진다.
- -1 또는 1에 가까운 값을 가질 수록 강한 상관관계를 나타낸다.

- 이변량 분석

- > 숫자 vs 숫자

- 상관관계를 이용하여 분석(corr)

```
corr = air.corr()
```

Out :

	Ozone	Solar.R	Wind	Temp
Ozone	1.000000	0.280068	-0.605478	0.683372
Solar.R	0.280068	1.000000	-0.056792	0.275840
Wind	-0.605478	-0.056792	1.000000	-0.457988
Temp	0.683372	0.275840	-0.457988	1.000000

> 판다스의 corr() 함수를 사용하면 모든 숫자 컬럼에 대한 상관 관계를 바로 확인할 수 있다.

- 이변량 분석

- > 숫자 vs 숫자

- 상관관계를 이용하여 분석(corr)

```
import seaborn as sns
sns.heatmap(corr, vmin=-1, vmax=1, annot=True)
```

Out :



- 사이킷런 소개와 특징

> 사이킷런(scikit-learn)은 파이썬에서 머신러닝 학습을 위한 라이브러리이다.

> <https://scikit-learn.org>

```
import sklearn  
print(sklearn.__version__)
```

Out : 1.4.2

• 머신러닝 따라하기

군집/회귀/분류 모델은 다르지만
머신러닝의 흐름은 동일하다.

> 붓꽃 품종 분류하기

- 꽃잎의 길이와 너비, 꽃받침의 길이와 너비 4개의 피쳐를 통해 붓꽃 품종을 분류한다.
- 종속변수는 아래의 3가지 품종의 종류이다.

iris setosa



petal sepal

iris versicolor



petal sepal

iris virginica



petal sepal

• 머신러닝 따라하기

- 붓꽃 품종 분류하기

> 라이브러리 호출

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

datasets은 사이킷런의 데이터 세트를 생성하는 모듈이다.

tree는 트리 기반 ML 알고리즘을 구현하는 모듈이다.

model_selection은 학습 데이터와 검증 데이터, 예측 데이터로 데이터를 분리하는 다양한 모듈이다.

train_test_split 함수는 데이터를 학습, 테스트 데이터로 분리하는 함수이다.

• 머신러닝 따라하기

- 붓꽃 품종 분류하기

> 데이터 살펴보기

```
import pandas as pd
iris = load_iris()
iris_data = iris.data
iris_label = iris.target
print('iris target값:', iris_label)
print('iris target명:', iris.target_names)
iris_df = pd.DataFrame(data=iris_data, columns = iris.feature_names)
iris_df['label'] = iris.target
iris_df.head(3)
```


- 머신러닝 따라하기

- 붓꽃 품종 분류하기

- > 학습, 테스트 데이터 나누기

```
X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,  
                                                    test_size = 0.2, random_state = 11)  
  
print(X_train[0])
```

Out : [5.1 3.5 1.4 0.2]

• 머신러닝 따라하기

- 붓꽃 품종 분류하기

> 학습하기

학습 / 예측

```
dt_clf = DecisionTreeClassifier(random_state = 11)
```

```
dt_clf.fit(X_train, y_train)
```

```
pred = dt_clf.predict(X_test)
```

평가

```
from sklearn.metrics import accuracy_score
```

```
acc = accuracy_score(y_test, pred)
```

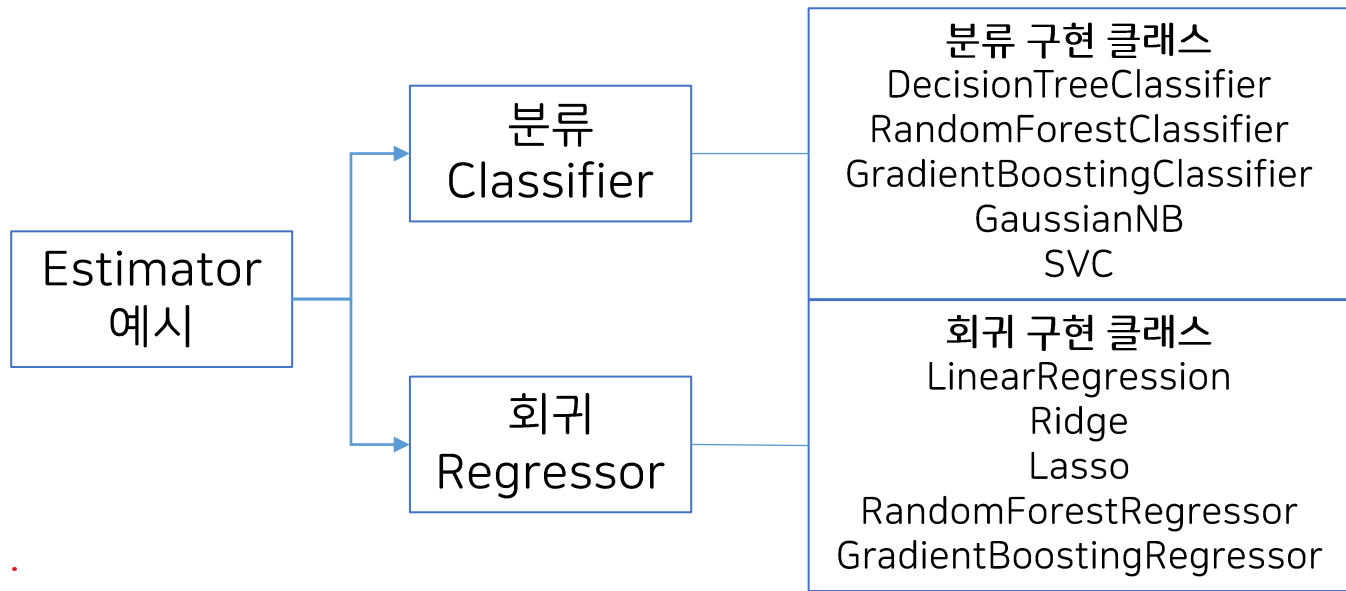
```
print('예측 정확도: {0:.4f}'.format(acc))
```

Out : 예측 정확도: 0.9333

• 사이킷런의 기반 프레임워크 익히기

> Estimator 이해

- 사이킷런은 일관성과 개발 편의성을 제공하기 위해 통일된 알고리즘의 클래스를 제공함.
- 이를 Estimator라고 한다.
- fit()과 predict() 또는 fit()과 transform()을 내장하고 있다.
- 이러한 Estimator는 다양한 모듈에 들어가 있고, 통일된 사용법을 제공한다.



• 사이킷런의 기반 프레임워크 익히기

> 사이킷런의 주요 모듈

분류	모듈명	설명
예제 데이터	datasets	예제로 제공하는 데이터 세트
preprocessing 피처 처리	★ preprocessing	데이터 전처리에 필요한 기능 제공
	feature_selection	피처를 우선순위대로 선택션 작업을 수행하는 다양한 기능 제공
	feature_extraction	텍스트 데이터나 이미지 데이터의 벡터화 기능 제공
피처 처리 & 차원 축소	decomposition	차원 축소와 관련한 알고리즘
데이터 분리, 검증 & 파라미터 튜닝	★ model_selection	학습에 도움이 되는 다양한 기능 제공
평가	metrics	학습된 모델에 대한 성능 측정 방법 제공
어디에 뭐가 있는지 알 필요는 있음. ML 알고리즘	ensemble	앙상블 알고리즘
	linear_model	선형 회귀 알고리즘
	naive_bayes	나이브 베이즈 알고리즘
	neighbors	최근접 이웃 알고리즘
	svm	서포트 벡터 머신 알고리즘
	tree	의사 결정 트리 알고리즘
	cluster	비지도 학습 알고리즘
유틸리티	pipeline	전처리, 학습, 예측 등 함께 묶어서 실행할 수 있는 유틸리티 제공

- 사이킷런의 기반 프레임워크 익히기

58

> load_datasets

```
from sklearn.datasets import load_iris
```

```
iris_data = load_iris()  
print(type(iris_data))
```

Out : <class 'sklearn.utils.Bunch'>

```
keys = iris_data.keys()  
print(keys)
```

Out : dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])

- 사이킷런의 기반 프레임워크 익히기

59

> train_test_split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size = 0.2, random_state = 11)

print('학습 데이터의 수:', len(X_train))
print('테스트 데이터의 수:', len(X_test))
```

Out : 학습 데이터의 수: 120
테스트 데이터의 수: 30

• 사이킷런의 기반 프레임워크 익히기

> 데이터 분할

기본적으로 학습:평가 3, 4:1이 좋다.

tsize=0.2정도.

기본적으로 학습데이터가 많아야 정확한 결과가 나옴.

학습용 데이터는 최대한 많을 수록 좋다.

- 모델 평가용 테스트 데이터와 구축용 데이터로 분할하여, 구축용 데이터로 모델을 생성하고 테스트 데이터로 모델이 얼마나 적합한지를 판단한다.

비율 궁금증 해결.



검증용

- 데이터의 양이 충분하지 않은 경우 다음과 같은 방법을 통해 데이터를 분할한다.

① 홀드아웃(hold-out)



검증 포기.

② K-fold 교차검증



검증데이터를 학습에도 활용.

검증용: 틈틈이 보는 쪽지 시험
-> 공부하지 않은 데이터로 시험

검정용: 최종적으로 평가하는 시험
수능

• 교차검증

1. 기존 방식은 학습 데이터 비중이 너무 적더라.

> 교차검증이란?

- 보통의 데이터는 학습, 검증 데이터로 모델을 검증한다.
- 하지만 고정된 테스트 데이터로만 검증을 반복하면 테스트 데이터에만 최적화되어 과적합이 발생할 수 있다.
- 이를 해결하기 위해 학습 데이터에서 검증 데이터를 교차로 분리하여 사용한다.

> 장점

- 모든 데이터셋을 훈련, 검증에 활용할 수 있다.
- 데이터 편향을 막을 수 있다.
- 적은 양의 데이터로도 정확하고 일반화된 성능 평가를 얻을 수 있다.

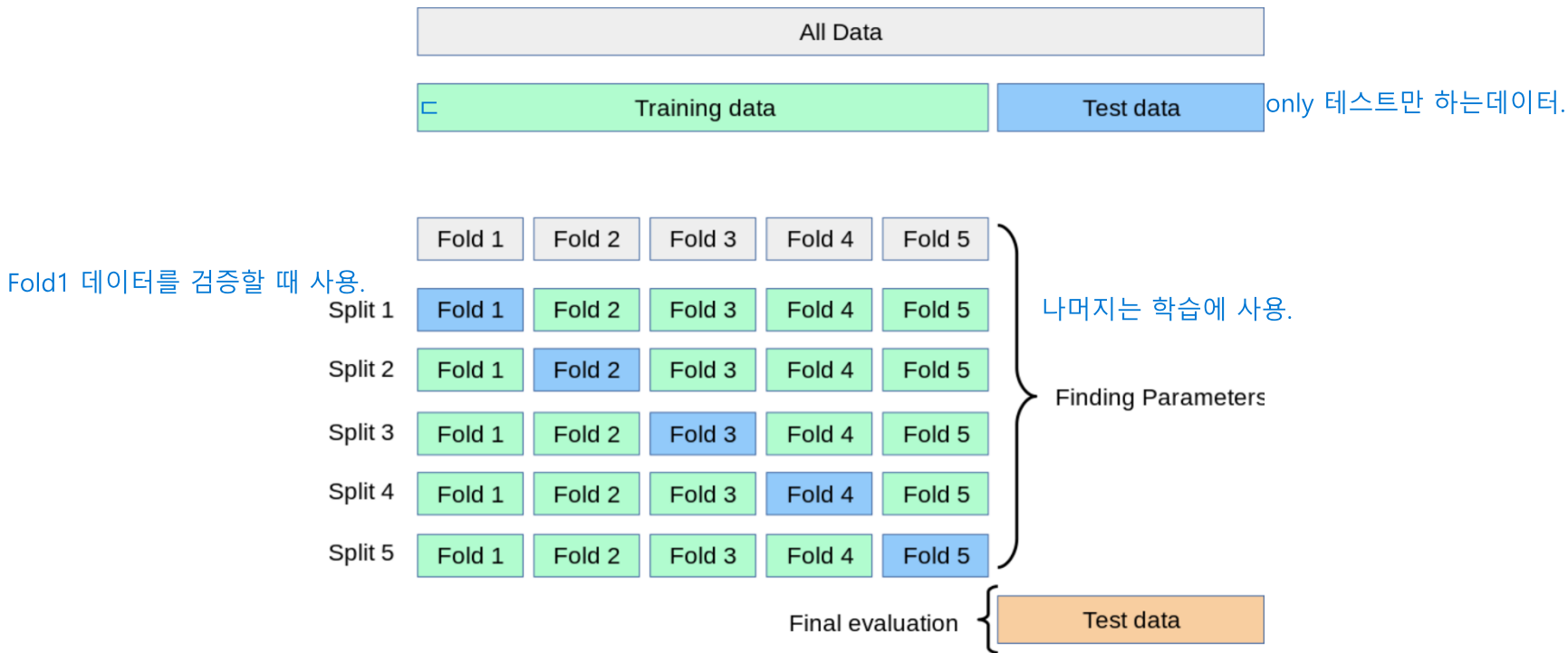
> 단점

- 학습 횟수가 증가하기때문에 오랜 시간이 걸린다.
- 데이터의 양과 분할 횟수가 늘어날수록 컴퓨터 자원과 계산 비용이 크게 증가한다.

• 교차검증

> K겹 교차 검증

- 보편적으로 많이 사용되는 교차 검증 기법
- K번 만큼 데이터를 나누어 돌아가며 학습, 검증 데이터로 사용



• 교차검증

> K겹 교차 검증 절차

- 전체 데이터셋을 학습, 테스트 데이터로 나눈다.
- 학습 데이터를 K개의 폴드로 나눈다.
- 첫 번째 폴드를 검증 데이터로 사용하고 나머지 데이터는 학습 데이터로 사용한다.
- 모델을 학습한 뒤, 검증 데이터로 평가한다.
- 차례대로 다음 폴드를 사용하여 반복한다.
- 총 K개의 성능 결과가 나오며, 이 K개의 평균을 해당 학습 모델의 성능이라고 한다.

- 교차검증

- > 교차검증 해보기

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import KFold
import numpy as np

iris = load_iris()
features = iris.data
label = iris.target
dt_clf = DecisionTreeClassifier(random_state = 156)
```

- 교차검증

- > 교차검증 해보기

```
kfold = KFold(n_splits = 5)
cv_accuracy = []
n_iter = 0

for train_index, test_index in kfold.split(features):
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]

    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
    n_iter += 1
```

- 교차검증

- > 교차검증 해보기

```
# 이전 코드에 이어서 작성
accuracy = np.round(accuracy_score(y_test, pred), 4)
train_size = X_train.shape[0]
test_size = X_test.shape[0]
print('#',n_iter)
print('교차 검증 정확도 :',accuracy)
print('학습데이터 크기 :',train_size)
print('검증데이터 크기 :',test_size)
print('검증 인덱스 :',test_index)
cv_accuracy.append(accuracy)

print('평균 검증 정확도 :', np.mean(cv_accuracy))
```

• 교차검증

> 교차검증 해보기

Out :

```
# 1
교차 검증 정확도 : 1.0
학습데이터 크기 : 120
검증데이터 크기 : 30
검증 인덱스 : [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29]

# 2
교차 검증 정확도 : 0.9667
학습데이터 크기 : 120
검증데이터 크기 : 30
검증 인덱스 : [30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
 54 55 56 57 58 59]

# 3
교차 검증 정확도 : 0.8667
학습데이터 크기 : 120
검증데이터 크기 : 30
검증 인덱스 : [60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
 84 85 86 87 88 89]

평균 검증 정확도 : 0.9
```

- 교차검증

- > 교차검증 해보기

```
import pandas as pd
```

```
iris = load_iris()
```

```
iris_df = pd.DataFrame(data = iris.data, columns = iris.feature_names)
```

```
iris_df['label'] = iris.target
```

```
iris_df.head(3)
```

Out :

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0

- 교차검증

- > 교차검증 해보기

```
kfold = KFold(n_splits = 3)
n_iter = 0
for train_index, test_index in kfold.split(iris_df):
    n_iter += 1
    label_train = iris_df['label'].iloc[train_index]
    label_test = iris_df['label'].iloc[test_index]
    print(f'#{n_iter} 교차검증')
    print('학습레이블 분포 :\n', label_train.value_counts())
    print('검증레이블 분포 :\n', label_test.value_counts())
```

• 교차검증

> 교차검증 해보기

Out :

#1 교차검증

학습레이블 분포 :

1 50

2 50

Name: label, dtype: int64

검증레이블 분포 :

0 50

Name: label, dtype: int64

#2 교차검증

학습레이블 분포 :

0 50

2 50

Name: label, dtype: int64

검증레이블 분포 :

1 50

Name: label, dtype: int64

#3 교차검증

학습레이블 분포 :

0 50

1 50

Name: label, dtype: int64

검증레이블 분포 :

2 50

Name: label, dtype: int64

- 교차검증

- > Stratified K-Fold

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits = 3)
n_iter = 0

for train_index, test_index in skf.split(iris_df, iris_df['label']):
    n_iter += 1
    label_train = iris_df['label'].iloc[train_index]
    label_test = iris_df['label'].iloc[test_index]
    print(f'#{n_iter} 교차검증')
    print('학습레이블 분포 :\\n', label_train.value_counts())
    print('검증레이블 분포 :\\n', label_test.value_counts())
```

- 교차검증

> Stratified K-Fold

Out :

```
#1 교차검증
학습레이블 분포 :
 2    34
0    33
1    33
Name: label, dtype: int64
검증레이블 분포 :
 0    17
1    17
2    16
Name: label, dtype: int64
#2 교차검증
학습레이블 분포 :
 1    34
0    33
2    33
Name: label, dtype: int64
검증레이블 분포 :
 0    17
2    17
1    16
Name: label, dtype: int64
```

```
#3 교차검증
학습레이블 분포 :
 0    34
1    33
2    33
Name: label, dtype: int64
검증레이블 분포 :
 1    17
2    17
0    16
Name: label, dtype: int64
```

- 교차검증

- > Stratified K-Fold

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import StratifiedKFold
import numpy as np

iris = load_iris()
features = iris.data
label = iris.target
dt_clf = DecisionTreeClassifier(random_state = 156)
```

- 교차검증

- > Stratified K-Fold

```
skf = StratifiedKFold(n_splits = 5)
cv_accuracy = []
n_iter = 0

for train_index, test_index in skf.split(features, label):
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]

    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
    n_iter += 1
```

- 교차검증

- > Stratified K-Fold

```
# 이전 코드에 이어서 작성
accuracy = np.round(accuracy_score(y_test, pred), 4)
train_size = X_train.shape[0]
test_size = X_test.shape[0]
print('#',n_iter)
print('교차 검증 정확도 : ',accuracy)
print('학습데이터 크기 : ',train_size)
print('검증데이터 크기 : ',test_size)
print('검증 인덱스 : ',test_index)
cv_accuracy.append(accuracy)

print('평균 검증 정확도 : ', np.mean(cv_accuracy))
```

- 교차검증

- > Stratified K-Fold

Out :

```
# 1
교차 검증 정확도 : 0.9667
학습데이터 크기 : 120
검증데이터 크기 : 30
검증 인덱스 : [ 0  1  2  3  4  5  6  7  8  9 50 51 52 53 54 55 56 57
58 59 100 101 102 103 104 105 106 107 108 109]

# 2
교차 검증 정확도 : 0.9667
학습데이터 크기 : 120
검증데이터 크기 : 30
검증 인덱스 : [ 10 11 12 13 14 15 16 17 18 19 60 61 62 63 64 65 66 67
68 69 110 111 112 113 114 115 116 117 118 119]

# 3
교차 검증 정확도 : 0.9
학습데이터 크기 : 120
검증데이터 크기 : 30
검증 인덱스 : [ 20 21 22 23 24 25 26 27 28 29 70 71 72 73 74 75 76 77
78 79 120 121 122 123 124 125 126 127 128 129]
```

평균 검증 정확도 : 0.9600200000000001

• 교차검증

- 교차 검증을 보다 간편하게 처리하기

> `cross_val_score()`

```
from sklearn.model_selection import cross_val_score

iris_data = load_iris()
dt_clf = DecisionTreeClassifier(random_state= 156)
data = iris_data.data
label = iris_data.target
scores = cross_val_score(dt_clf, data, label, scoring = 'accuracy', cv = 5)
print('교차 검증별 정확도: ', np.round(scores, 4))
print('평균 검증 정확도: ', np.round(np.mean(scores), 4))
```

Out : 교차 검증별 정확도: [0.9667 0.9667 0.9 0.9667 1.]

평균 검증 정확도: 0.96

• 교차검증

- 교차 검증을 포다 간편하게 처리하기

78

> cross_validate()

```
from sklearn.model_selection import cross_validate, cross_val_predict
```

```
cross_validate(dt_clf, data, label, scoring = ['accuracy', 'roc_auc_ovo'], cv = 5)
```

```
Out: {'fit_time': array([0.00099707, 0.00099063, 0.          , 0.00099993, 0.          ]),  
      'score_time': array([0.00401044, 0.00200033, 0.00200081, 0.00200033, 0.00200057]),  
      'test_accuracy': array([0.96666667, 0.96666667, 0.9          , 0.96666667, 1.          ]),  
      'test_roc_auc_ovo': array([0.975, 0.975, 0.925, 0.975, 1.          ])}
```

> cross_val_predict()

```
cross_val_predict(dt_clf, data, label, cv = 5)
```

```
Out: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
            0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
            1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2,  
            2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2,  
            2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

What is an Hyper Parameter?

기존: 모델에 사용되는 파라미터는 랜덤한 값이 들어가기도 한다.

하이퍼 파라미터: 사용자가 지정한 custom parameter 기반으로

모델의 학습 방향을 변경한다.

• 하이퍼 파라미터

Grid Search가 뭘까?

검증 데이터 != 테스트 데이터

검증 데이터는 학습+검증 중간단계에서 사용.

GridSearch는 여러 하이퍼 파라미터의

모든 경우의 수 중 가장 좋은 데이터 페어를 찾는다.

-> 경우의 수가 많아 시간이 오래 걸린다.

테스트 데이터는 마지막에 최종 1번 사용.

> GridSearchCV

```
from sklearn.model_selection import GridSearchCV
```

학습모델이 아니라서 반드시
다른 학습 모델을 사용해야함.

```
iris = load_iris()
```

```
X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,  
                                                    test_size=0.2, random_state=121)
```

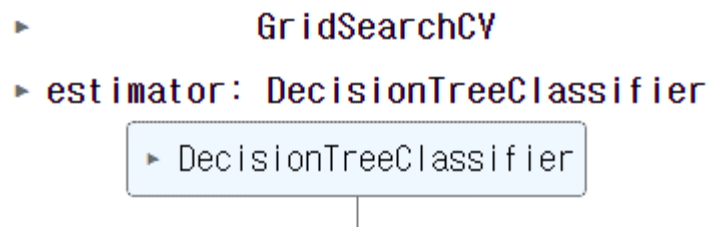
```
dtree = DecisionTreeClassifier()
```

```
parameters = {'max_depth':[1,2,3], 'min_samples_split':[2,3]}
```

```
grid_dtree = GridSearchCV(dtree, param_grid=parameters, cv=3, refit=True)
```

```
grid_dtree.fit(X_train, y_train)
```

Out :



- 하이퍼 파라미터

80

> GridSearchCV

```
import pandas as pd
```

```
scores_df = pd.DataFrame(grid_dtree.cv_results_)
scores_df[['params', 'mean_test_score', 'rank_test_score',
           'split0_test_score', 'split1_test_score', 'split2_test_score']]
```

Out :

	params	mean_test_score	rank_test_score	split0_test_score	split1_test_score	split2_test_score
0	{'max_depth': 1, 'min_samples_split': 2}	0.700000	5	0.700	0.7	0.70
1	{'max_depth': 1, 'min_samples_split': 3}	0.700000	5	0.700	0.7	0.70
2	{'max_depth': 2, 'min_samples_split': 2}	0.958333	3	0.925	1.0	0.95
3	{'max_depth': 2, 'min_samples_split': 3}	0.958333	3	0.925	1.0	0.95
4	{'max_depth': 3, 'min_samples_split': 2}	0.975000	1	0.975	1.0	0.95
5	{'max_depth': 3, 'min_samples_split': 3}	0.975000	1	0.975	1.0	0.95

- 하이퍼 파라미터

81

> GridSearchCV

```
print('GridSearchCV 최적 파라미터:', grid_dtree.best_params_)  
print(f'GridSearchCV 최고 정확도: {grid_dtree.best_score_:.4f}')
```

Out : GridSearchCV 최적 파라미터: {'max_depth': 3,
 'min_samples_split': 2}
GridSearchCV 최고 정확도: 0.9750

```
estimator = grid_dtree.best_estimator_  
pred = estimator.predict(X_test)  
print(f'테스트 데이터 세트 정확도:{accuracy_score(y_test,pred):.4f}')
```

Out : 테스트 데이터 세트 정확도: 0.9667

- 하이퍼 파라미터

- > GridSearchCV

```
from sklearn.model_selection import GridSearchCV
params = {'max_depth': [ 6, 8, 10, 12, 16, 20, 24]}
grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5,
verbose=1 )
grid_cv.fit(X_train , y_train)
print(f'GridSearchCV 최고 정확도 수치:{grid_cv.best_score_: .4f}')
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)
```

Out : split=5 Fitting 5 folds for each of 검증 7번 7 candidates, totalling 총 35번의 학습 횟수. 35 fits
GridSearchCV 최고 정확도 수치: 0.9500
GridSearchCV 최적 하이퍼 파라미터: {'max_depth': 6}

- 하이퍼 파라미터

83

> GridSearchCV

```
cv_results_df = pd.DataFrame(grid_cv.cv_results_)
```

```
cv_results_df[['param_max_depth', 'mean_test_score']]
```

Out :

	param_max_depth	mean_test_score
0	6	0.95
1	8	0.95
2	10	0.95
3	12	0.95
4	16	0.95
5	20	0.95
6	24	0.95

- 하이퍼 파라미터

84

- > 최적의 하이퍼 파라미터 튜닝

```
max_depths = [ 6, 8 ,10, 12, 16 ,20, 24]
for depth in max_depths:
    dt_clf = DecisionTreeClassifier(max_depth=depth, min_samples_split=16,
                                    random_state=156)
    dt_clf.fit(X_train , y_train)
    pred = dt_clf.predict(X_test)
    accuracy = accuracy_score(y_test , pred)
    print(f'max_depth = {depth} 정확도: {accuracy:.4f}')
```

Out : max_depth = 6 정확도: 0.9667
max_depth = 8 정확도: 0.9667
max_depth = 10 정확도: 0.9667
max_depth = 12 정확도: 0.9667
max_depth = 16 정확도: 0.9667
max_depth = 20 정확도: 0.9667
max_depth = 24 정확도: 0.9667

- 하이퍼 파라미터

85

- > 최적의 하이퍼 파라미터 튜닝

```
params = {'max_depth' : [ 8 , 12, 16 ,20],  
          'min_samples_split' : [16, 24],}  
  
grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5,  
                        verbose=1 )  
grid_cv.fit(X_train , y_train)  
print(f'GridSearchCV 최고 정확도 수치: {grid_cv.best_score_:.4f}')  
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)
```

Out : Fitting 5 folds for each of 8 candidates, totalling 40 fits
GridSearchCV 최고 정확도 수치: 0.9667
GridSearchCV 최적 하이퍼 파라미터: {'max_depth': 8, 'min_samples_split': 16}

- 하이퍼 파라미터

- > 최적의 하이퍼 파라미터 튜닝

```
best_df_clf = grid_cv.best_estimator_  
pred1 = best_df_clf.predict(X_test)  
accuracy = accuracy_score(y_test , pred1)  
print(f'결정 트리 예측 정확도:{accuracy:.4f}')
```

Out : 결정 트리 예측 정확도:0.9667

• 데이터 전처리

> 레이블 인코딩

```
from sklearn.preprocessing import LabelEncoder

items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '선풍기', '믹서', '냉장고']

encoder = LabelEncoder()
encoder.fit(items)
labels = encoder.transform(items)
print('인코딩 변환값:', labels)
```

Out : 인코딩 변환값 : [0 1 4 5 3 3 2 1]

- 데이터 전처리

- > 레이블 인코딩

```
print('인코딩 클래스 :', encoder.classes_)
```

Out : 인코딩 클래스 : ['TV' '냉장고' '믹서' '선풍기' '전자레인지' '컴퓨터']

```
print('디코딩 :', encoder.inverse_transform([4, 5, 2, 1]))
```

Out : 디코딩 : ['전자레인지' '컴퓨터' '믹서' '냉장고']

- 데이터 전처리

- > 원-핫 인코딩

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '선풍기', '믹서', '냉장고']

items = np.array(items).reshape(-1, 1)

oh_encoder = OneHotEncoder()
oh_encoder.fit(items)
oh_labels = oh_encoder.transform(items)
```

• 데이터 전처리

> 원-핫 인코딩

```
print('원-핫 인코딩 데이터')  
print(oh_labels.toarray())  
print('원-핫 인코딩 데이터 크기')  
print(oh_labels.shape)
```

Out :

```
원-핫 인코딩 데이터  
[[1.  0.  0.  0.  0.  0.]  
 [0.  1.  0.  0.  0.  0.]  
 [0.  0.  0.  0.  1.  0.]  
 [0.  0.  0.  0.  0.  1.]  
 [0.  0.  0.  1.  0.  0.]  
 [0.  0.  0.  1.  0.  0.]  
 [0.  0.  1.  0.  0.  0.]  
 [0.  1.  0.  0.  0.  0.]]  
원-핫 인코딩 데이터 크기  
(8, 6)
```

• 데이터 전처리

> 원-핫 인코딩

```
import pandas as pd

df = pd.DataFrame({'item':['TV', '냉장고', '전자레인지', '컴퓨터',  
                           '선풍기', '선풍기', '믹서', '냉장고']})

pd.get_dummies(df)
```

Out :

	item_TV	item_냉장고	item_믹서	item_선풍기	item_전자레인지	item_컴퓨터
0	1	0	0	0	0	0
1	0	1	0	0	0	0
2	0	0	0	0	1	0
3	0	0	0	0	0	1
4	0	0	0	1	0	0
5	0	0	0	1	0	0
6	0	0	1	0	0	0
7	0	1	0	0	0	0

• 데이터 전처리

> 피쳐 스케일링과 정규화

- 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업

> 표준화

- 평균이 0, 분산이 1인 가우시안 정규 분포를 가진 값으로 변환

$$x_{i_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

> 정규화

- 값을 범위를 모두 0 ~ 1의 값으로 변환

$$x_{i_new} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- 데이터 전처리

- > StandardScaler

- 표준화

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
iris_data = iris.data
iris_df = pd.DataFrame(data = iris_data, columns = iris.feature_names)

print('feature 들의 평균 값 :', iris_df.mean())
print('feature 들의 분산 값 :', iris_df.var())
```

- 데이터 전처리

- > StandardScaler

- 표준화

Out :

```
feature 들의 평균 값 :
  sepal length (cm)    5.843333
  sepal width (cm)     3.057333
  petal length (cm)    3.758000
  petal width (cm)     1.199333
dtype: float64
feature 들의 분산 값 :
  sepal length (cm)    0.685694
  sepal width (cm)     0.189979
  petal length (cm)    3.116278
  petal width (cm)     0.581006
dtype: float64
```

- 데이터 전처리

- > StandardScaler

- 표준화

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

iris_df_scaled = pd.DataFrame(data = iris_scaled, columns = iris.feature_names)

print('feature 들의 평균 값 :\n', iris_df_scaled.mean())
print('feature 들의 분산 값 :\n', iris_df_scaled.var())
```

- 데이터 전처리

- > StandardScaler

- 표준화

Out :

```
feature 들의 평균 값 :
  sepal length (cm)    -1.690315e-15
  sepal width (cm)     -1.842970e-15
  petal length (cm)    -1.698641e-15
  petal width (cm)     -1.409243e-15
dtype: float64
feature 들의 분산 값 :
  sepal length (cm)     1.006711
  sepal width (cm)      1.006711
  petal length (cm)     1.006711
  petal width (cm)      1.006711
dtype: float64
```

- 데이터 전처리

- > MinMaxScaler

- 정규화

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

iris_df_scaled = pd.DataFrame(data = iris_scaled, columns = iris.feature_names)

print('feature 들의 최소 값 :\\n', iris_df_scaled.min())
print('feature 들의 최대 값 :\\n', iris_df_scaled.max())
```

- 데이터 전처리

- > MinMaxScaler

- 정규화

Out :

```
feature 들의 최소 값 :
  sepal length (cm)    0.0
  sepal width (cm)     0.0
  petal length (cm)    0.0
  petal width (cm)     0.0
dtype: float64
feature 들의 최대 값 :
  sepal length (cm)    1.0
  sepal width (cm)     1.0
  petal length (cm)    1.0
  petal width (cm)     1.0
dtype: float64
```

- 회귀란?

- > 여러 개의 독립 변수와 한 개의 종속변수 간의 상관관계 모델링
- > $Y = W_0X_0 + W_1X_1 + W_2X_2 + \dots + W_NX_N$ 일 경우
 - Y : 종속변수, 레이블
 - X : 독립변수, 피쳐
 - W : 회귀 계수
- > 최적의 회귀 계수를 찾아내는 일이 회귀 분석이다.
- > 독립 변수가 1개면 단일 회귀, 다수면 다중 회귀로 나뉜다.
- > 회귀 계수의 결합에 따라 선형이냐 비선형이냐로 나뉜다.

• 회귀란?

> 선형 결합이란?

- 회귀 계수를 선형 모델로 만들 수 있는 상태

- $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 \Rightarrow \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$

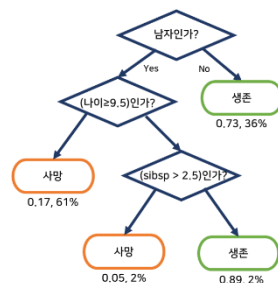
- $y = \beta_0 x^{\beta_1} \Rightarrow \log(y) = \log(\beta_0 x^{\beta_1}) \Rightarrow \log \beta_0 + \beta_1 \log(x) \Rightarrow y^* = \beta_0^* + \beta_1 x^*$

- $y = \frac{e^{\beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3}}{1 + e^{\beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3}} \Rightarrow \frac{y}{1-y} = e^{\beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3} \Rightarrow \log\left(\frac{y}{1-y}\right) = y^* = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$

> 비선형 결합이란?

- 선형 모델로 표현이 불가능한 상태

$$y = \frac{\beta_1 x}{\beta_2 + x}$$



- 회귀란?

- > 대표적인 선형 회귀

- 일반 선형 회귀 : 예측값과 실제값의 오류를 최소화할 수 있도록 회귀 계수를 최적화하며, 규제(Regularization)를 적용하지 않는 모델
- 릿지(Ridge) : 선형 회귀에 L2 규제를 추가한 모델
- 라쏘(Lasso) : 선형 회귀에 L1 규제를 추가한 모델
- 엘리스틱넷 : L2, L1 규제를 결합한 모델
- 로지스틱 회귀 : 회귀보단 이진 분류에 특화된 모델
- SGD : 확률적 경사 하강법을 적용한 선형 모델

- 회귀 평가 지표

- > 오류의 차이를 판단하는 평가 지표

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
MSLE	MSE에 로그를 적용한 것입니다. 결정값이 클 수록 오류값도 커지기 때문에 일부 큰 오류값들로 인해 전체 오류값이 커지는 것을 막아줍니다.	$\text{Log}(MSE)$
RMSE	MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
RMSLE	RMSE에 로그를 적용한 것입니다. 결정값이 클 수록 오류값도 커지기 때문에 일부 큰 오류값들로 인해 전체 오류값이 커지는 것을 막아줍니다.	$\text{Log}(RMSE)$
R ²	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 Variance}}{\text{실제 값 Variance}}$

• 회귀 평가 지표

103

> MSE (Mean Squared Error)

- 예측 값과 실제 값의 차이에 대한 제곱에 대하여 평균을 낸 값
- MSE 오차가 작으면 작을수록 좋지만, 과대적합이 될 수 있음에 주의합니다.
- 예측 값과 실제 값보다 크게 예측이 되는지 작게 예측되는지 알 수 없습니다.

> MAE (Mean Absolute Error)

- 예측값과 실제값의 차이에 대한 절대값에 대하여 평균을 낸 값
- 실제 값과 예측 값 차이를 절대 값으로 변환해 평균을 계산합니다. 작을수록 좋지만, 과대적합이 될 수 있음에 주의합니다.
- 스케일에 의존적입니다.

• 회귀 평가 지표

104

> RMSE (Root Mean Squared Error)

- 예측값과 실제값의 차이에 대한 제곱에 대하여 평균을 낸 뒤 루트를 씌운 값
- MSE의 장단점을 거의 그대로 따라갑니다.
- 제곱 오차에 대한 왜곡을 줄여줍니다.

> R2 Score (결정계수)

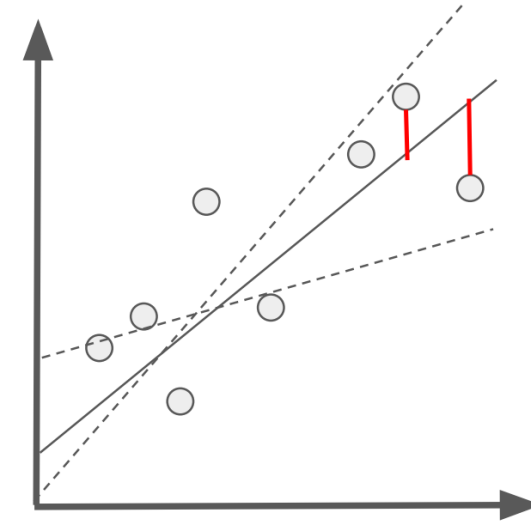
- 통계학 회귀분석에서 자주 쓰이는 회귀 평가 지표.
- 실제 값의 분산 대비 예측 값의 분산 비율을 나타냅니다.
- 1에 가까울 수록 좋은 모델, 0에 가까울 수록 나쁨, 음수가 나오면 잘못 평가 되었음을 의미합니다.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

• 단순 선형 회귀

> 독립변수 하나, 종속변수 하나인 선형 회귀

- $Y = W_0 + W_1 * x$
- W_0 : 회귀 계수
- W_1 : 절편
- 오류(잔차) : 실제 값과 회귀 예측값의 차이



> 회귀계수와 절편은 잔차의 제곱합(RSS)이 최소가 되는 값이다.

- 최소제곱법(OLS) 또는 경사하강법을 통해 계산한다.

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

- 선형 회귀 분석

- > 당뇨병 수치 예측

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from scipy import stats
from sklearn.datasets import load_diabetes

diabetes = load_diabetes()

diabetes_df = pd.DataFrame(diabetes.data , columns = diabetes.feature_names)
diabetes_df['target'] = diabetes.target
```

- 선형 회귀 분석

> 당뇨병 수치 예측

```
diabetes_df.head()
```

Out :

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

- 선형 회귀 분석

- > 당뇨병 수치 예측

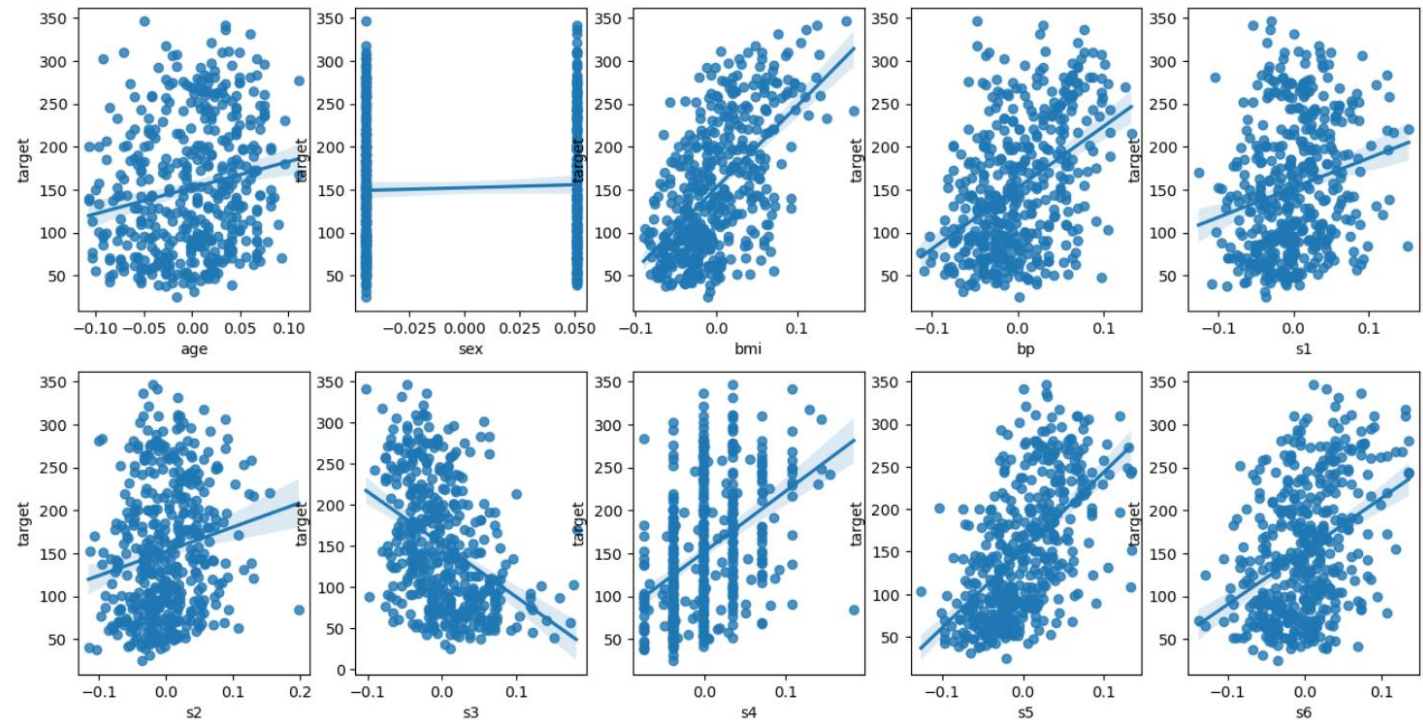
```
fig, axs = plt.subplots(figsize=(16,8) , ncols=5 , nrow=2)
lm_features = ['age','sex','bmi','bp','s1','s2','s3','s4','s5','s6']

for i , feature in enumerate(lm_features):
    row = int(i/5)
    col = i%5
    sns.regplot(x=feature , y='target',data=diabetes_df, ax=axs[row][col])
diabetes_df.corr()['target']
```

• 선형 회귀 분석

> 당뇨병 수치 예측

```
age      0.187889
sex      0.043062
bmi      0.586450
bp       0.441482
s1       0.212022
s2       0.174054
s3      -0.394789
s4       0.430453
s5       0.565883
s6       0.382483
target   1.000000
Name: target, dtype: float64
```



- 선형 회귀 분석

- > 당뇨병 수치 예측

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

y = diabetes_df.iloc[:, -1]
X = diabetes_df.iloc[:, :-1]
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3, random_state=156)
```

- 선형 회귀 분석

> 당뇨병 수치 예측

```
lr = LinearRegression()  
lr.fit(X_train ,y_train )  
pred = lr.predict(X_test)  
mse = mean_squared_error(y_test, pred)  
rmse = np.sqrt(mse)  
print(f'MSE : {mse:.3f} , RMSE : {rmse:.3F}')
```

```
print(f'Variance score : {r2_score(y_test, pred):.3f}')
```

Out : MSE : 2993.705 , RMSE : 54.715
Variance score : 0.497

• 선형 회귀 분석

112

> 당뇨병 수치 예측

```
print('절편 값:', lr.intercept_)  
print('회귀 계수값:', np.round(lr.coef_, 1))
```

Out : 절편 값: 152.38617209733573

회귀 계수값: [39.1 -251.9 468.8 305.3 -1146.9 788. 177.2 117.4
937.9 53.7]

```
coeff = pd.Series(data=np.round(lr.coef_, 1), index=X.columns )  
coeff
```

Out :

age	39.1
sex	-251.9
bmi	468.8
bp	305.3
s1	-1146.9
s2	788.0
s3	177.2
s4	117.4
s5	937.9
s6	53.7
dtype: float64	

- 선형 회귀 분석

- > 당뇨병 수치 예측

```
from sklearn.model_selection import cross_val_score
lr = LinearRegression()

neg_mse_scores = cross_val_score(lr, X, y, scoring="neg_mean_squared_error", cv = 5)
# 'neg_root_mean_squared_error', 'neg_mean_absolute_error', 'r2'

rmse_scores = np.sqrt(-1 * neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

print('5 folds 의 개별 Negative MSE scores:', np.round(neg_mse_scores, 2))
print('5 folds 의 개별 RMSE scores :', np.round(rmse_scores, 2))
print(f'5 folds 의 평균 RMSE : {avg_rmse:.3f}')
```

- 선형 회귀 분석

114

- > 당뇨병 수치 예측

Out : 5 folds 의 개별 Negative MSE scores: [-2779.92 -3028.84 -3237.69 -3008.75 -2910.21]
5 folds 의 개별 RMSE scores : [52.72 55.03 56.9 54.85 53.95]
5 folds 의 평균 RMSE : 54.692

- 다항회귀

- > 다항회귀 실습

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np
```

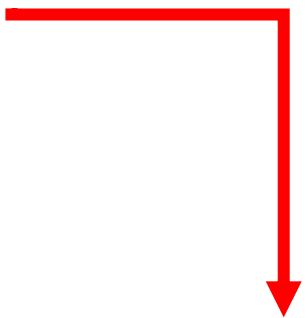
```
X = np.arange(4).reshape(2,2)
print('일차 단항식 계수 feature:\n',X )
poly = PolynomialFeatures(degree=2)
poly.fit(X)
poly_ftr = poly.transform(X)
print('변환된 2차 다항식 계수 feature:\n', poly_ftr)
```

• 다항회귀

> 다항회귀 실습

x_1	x_2
0	1
2	3

1차 식의 데이터를
2차 식의 데이터 변환하는 방식
* bias라는 조건이 붙는다



bias	x_1	x_2	x_1^2	x_1x_2	x_2^2
1	0	1	0	0	1
1	2	3	4	6	9

- 다항회귀

117

- > 다항회귀 실습

```
def polynomial_func(X):  
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3  
    return y
```

```
X = np.arange(0,4).reshape(2,2)  
print('일차 단항식 계수 feature: %n' ,X)  
y = polynomial_func(X)  
print('삼차 다항식 결정값: %n', y)
```

Out : 일차 단항식 계수 feature:
[[0 1]
 [2 3]]
삼차 다항식 결정값:
[5 125]

- 다항회귀

> 다항회귀 실습

```
poly_ftr = PolynomialFeatures(degree=3).fit_transform(X)
print('3차 다항식 계수 feature: \n',poly_ftr)
```

Out : 3차 다항식 계수 feature:

```
[[ 1.  0.  1.  0.  0.  1.  0.  0.  0.  1.]
 [ 1.  2.  3.  4.  6.  9.  8. 12. 18. 27.]]
```

> 다음과 같은 규칙을 가지고 생성한다.

x_1	x_2		bias	x_1	x_2	x_1^2	x_1x_2	x_2^2	x_1^3	$x_1^2x_2$	$x_1x_2^2$	x_2^3
0	1		1	0	1	0	0	1	0	0	0	1
2	3		1	2	3	4	6	9	8	12	18	27

- 다항회귀

119

- > 다항회귀 실습

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(poly_ftr,y)
```

```
print('Polynomial 회귀 계수\\n' , np.round(model.coef_, 2))
```

```
print('Polynomial 회귀 Shape :', model.coef_.shape)
```

Out : Polynomial 회귀 계수

[0. 0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]

Polynomial 회귀 Shape : (10,)

- 다항회귀와 과적합/과소적합

120

- > 과적합, 과소적합 이해

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

def true_fun(X):
    return np.cos(1.5 * np.pi * X)

np.random.seed(0)
n_samples = 30
X = np.sort(np.random.rand(n_samples))
y = true_fun(X) + np.random.randn(n_samples) * 0.1
```

- 다항회귀와 과적합/과소적합

121

- > 과적합, 과소적합 이해

```
plt.figure(figsize=(14, 5))
degrees = [1, 4, 15]

for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i + 1)
    polynomial_features = PolynomialFeatures(degree=degrees[i], include_bias=False)
    lr_reg = LinearRegression()
    X_poly = polynomial_features.fit_transform(X.reshape(-1,1), y)
    lr_reg.fit(X_poly, y)
    scores = cross_val_score(lr_reg, X_poly, y, scoring='neg_mean_squared_error', cv=10)
    coeff = lr_reg.coef_
```

- 다항회귀와 과적합/과소적합

- > 과적합, 과소적합 이해

```
# 이전 코드에 이어서 작성
print(f'\nDegree {degrees[i]} 회귀계수는 {np.round(coeff, 2)}\n입니다.')
print(f'MSE는 {-1 * np.mean(scores)}입니다.')
X_test = np.linspace(0, 1, 100).reshape(-1, 1)
X_test_poly = polynomial_features.transform(X_test)
pred = lr_reg.predict(X_test_poly)
plt.plot(X_test, pred, label = 'Model')
plt.plot(X_test, true_fun(X_test), '--', label='True Function')
plt.scatter(X, y, edgecolors='b', s = 20, label = 'Samples')
plt.xlabel('x');plt.ylabel('y'); plt.xlim((0,1)); plt.ylim((-2, 2)); plt.legend()
plt.title(f'Degree {degrees[i]}\nMSE={-scores.mean():.2f}')
plt.show()
```

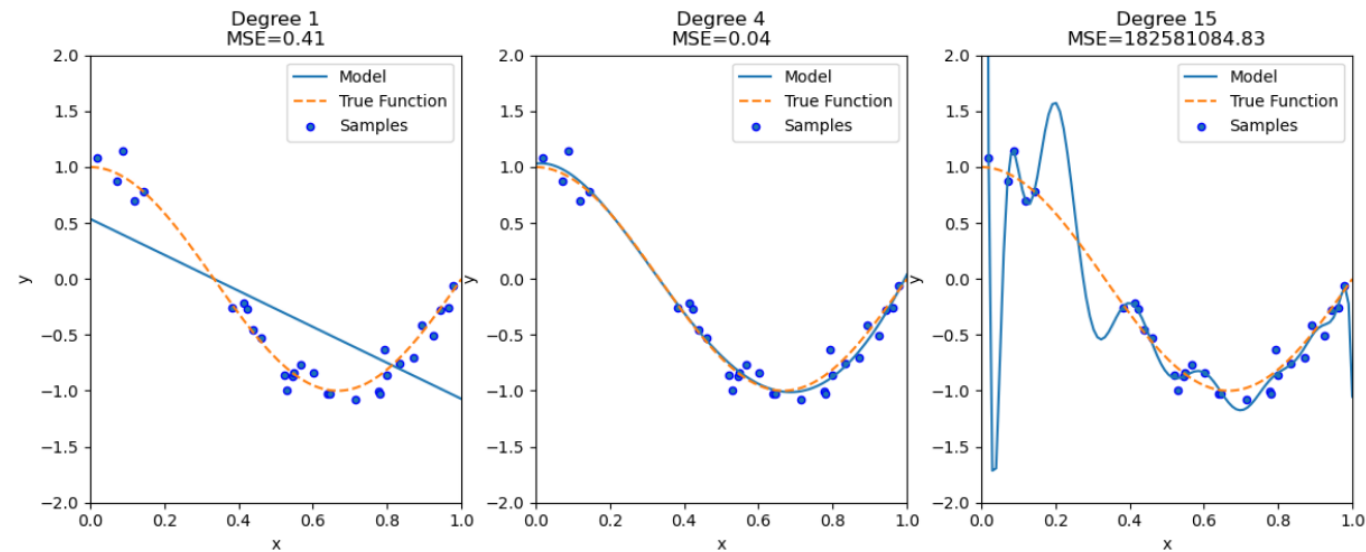
• 다항회귀와 과적합/과소적합

> 과적합, 과소적합 이해

Out : Degree 1 회귀계수는
[-1.61]
입니다.
MSE는 0.40772896250986845입니다.

Degree 4 회귀계수는
[0.47 -17.79 23.59 -7.26]
입니다.
MSE는 0.0432087498723184입니다.

Degree 15 회귀계수는
[-2.98294000e+03 1.03899850e+05 -1.87416981e+06 2.03717199e+07
-1.44874017e+08 7.09319141e+08 -2.47067173e+09 6.24564702e+09
-1.15677216e+10 1.56895933e+10 -1.54007040e+10 1.06457993e+10
-4.91381016e+09 1.35920643e+09 -1.70382078e+08]
입니다.
MSE는 182581084.8263125입니다.



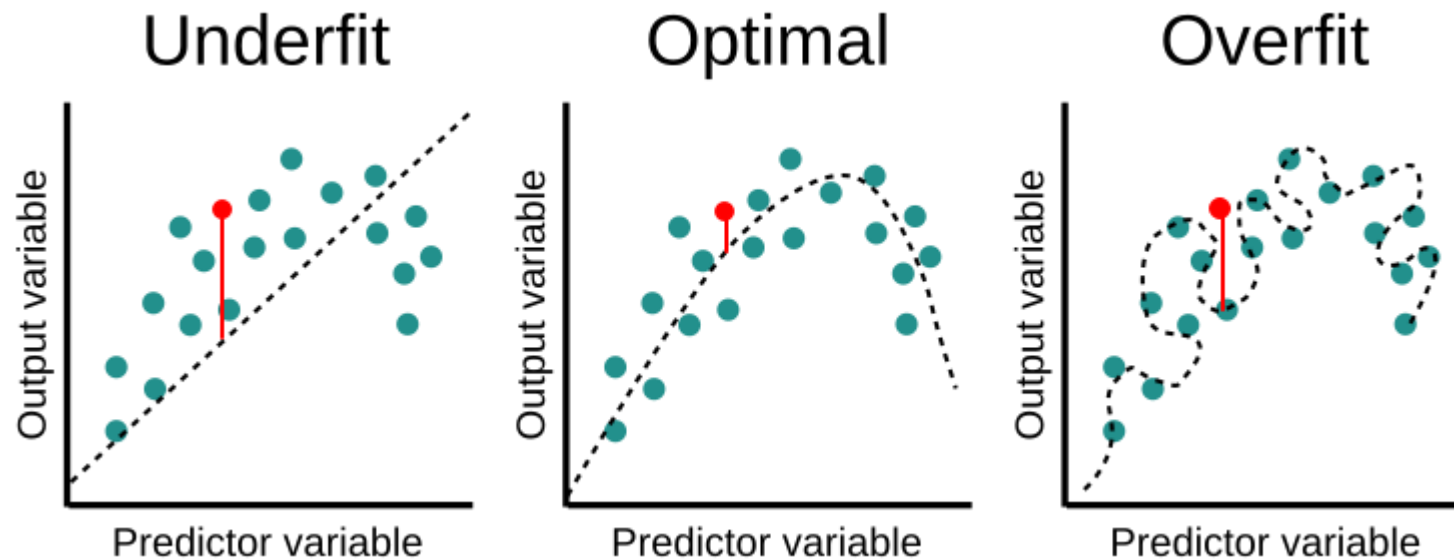
- 과적합/과소적합

- > 과적합

- 데이터를 학습할 때, 학습 데이터에만 과하게 학습을 하여 일반화하지 못하게 되어 테스트 데이터에서 오차가 커지는 현상

- > 과소적합

- 학습 데이터에 대한 학습이 부족하여 훈련/테스트 데이터에서 오차가 크게 나오는 현상



- 규제 선형 모델

125

- > 릿지 회귀 : L2 규제

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

y = diabetes_df.iloc[:, -1]
X = diabetes_df.iloc[:, :-1]
ridge = Ridge(alpha = 10)
neg_mse_scores = cross_val_score(ridge, X, y, scoring="neg_mean_squared_error", cv = 5)
rmse_scores = np.sqrt(-1 * neg_mse_scores)
avg_rmse = np.mean(rmse_scores)
print(' 5 folds 의 개별 Negative MSE scores: ', np.round(neg_mse_scores, 3))
print(' 5 folds 의 개별 RMSE scores : ', np.round(rmse_scores, 3))
print(' 5 folds 의 평균 RMSE : {0:.3f} '.format(avg_rmse))
```

• 규제 선형 모델

> 릿지 회귀 : L2 규제

Out : 5 folds 의 개별 Negative MSE scores: [-4540.477 -5430.505 -5284.404 -4364.15 -5463.355]
5 folds 의 개별 RMSE scores : [67.383 73.692 72.694 66.062 73.915]
5 folds 의 평균 RMSE : 70.749

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

- 규제 선형 모델

127

> 릿지 회귀 : L2 규제

```
alphas = [0, 0.1, 1, 10, 100]
for alpha in alphas :
    ridge = Ridge(alpha = alpha)

    neg_mse_scores = cross_val_score(ridge, X, y,
                                     scoring="neg_mean_squared_error", cv = 5)

    avg_rmse = np.mean(np.sqrt(-1 * neg_mse_scores))
    print(f'alpha {alpha} 일 때 5 folds 의 평균 RMSE : {avg_rmse:.3f}')
```

Out : alpha 0 일 때 5 folds 의 평균 RMSE : 54.692
alpha 0.1 일 때 5 folds 의 평균 RMSE : 54.825
alpha 1 일 때 5 folds 의 평균 RMSE : 58.449
alpha 10 일 때 5 folds 의 평균 RMSE : 70.749
alpha 100 일 때 5 folds 의 평균 RMSE : 76.399

- 규제 선형 모델

128

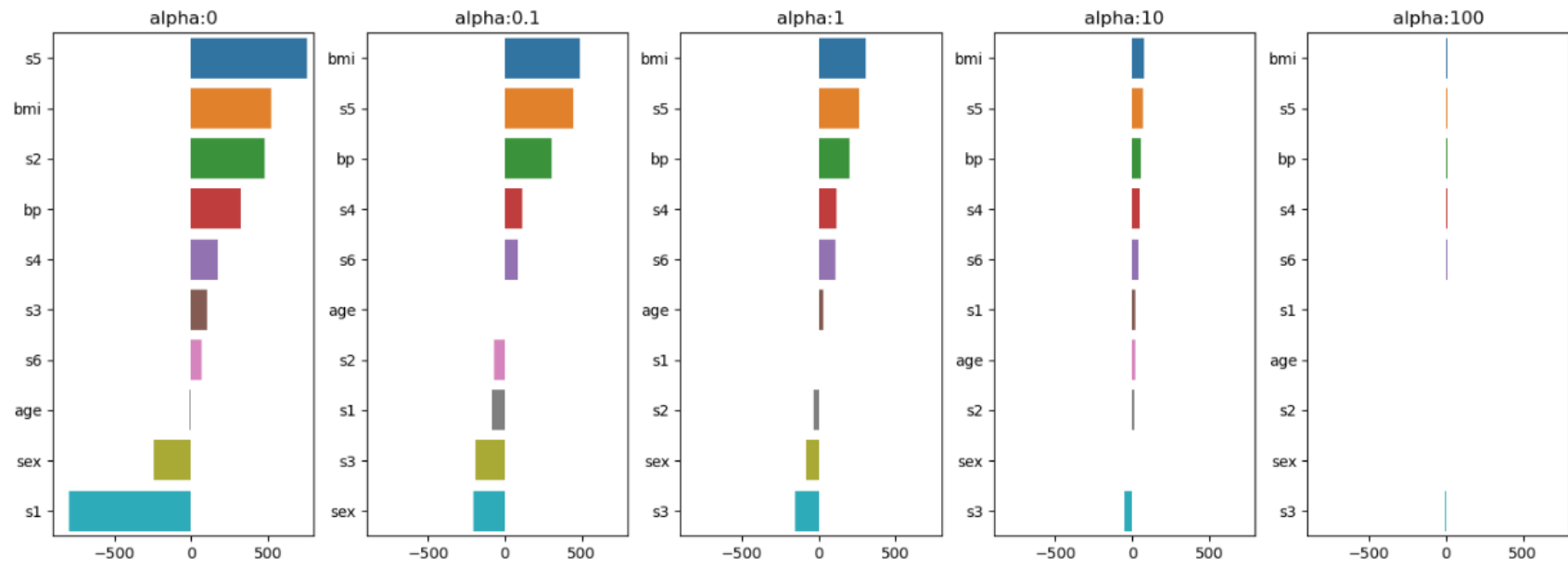
- > 릿지 회귀 : L2 규제

```
fig, axs = plt.subplots(figsize = (18,6), nrows=1, ncols=5)
coeff_df = pd.DataFrame()
for pos, alpha in enumerate(alphas):
    ridge = Ridge(alpha = alpha)
    ridge.fit(X, y)
    coeff = pd.Series(data = ridge.coef_, index = X.columns)
    colname='alpha:'+ str(alpha)
    coeff_df[colname] = coeff
    coeff = coeff.sort_values(ascending = False)
    axs[pos].set_title(colname); axs[pos].set_xlim(-900, 800)
    sns.barplot(x = coeff.values, y=coeff.index, ax=axs[pos])
plt.show()
```

• 규제 선형 모델

> 릿지 회귀 : L2 규제

Out :



- 규제 선형 모델

130

> 릿지 회귀 : L2 규제

```
ridge_alphas = [0, 0.1, 1, 10, 100]
sort_column = 'alpha:' + str(ridge_alphas[0])
coeff_df.sort_values(by=sort_column, ascending=False)
```

Out :

	alpha:0	alpha:0.1	alpha:1	alpha:10	alpha:100
s5	751.273700	443.812917	262.944290	70.143948	8.876851
bmi	519.845920	489.695171	306.352680	75.416214	9.240720
s2	476.739021	-70.826832	-29.515495	13.948715	2.616766
bp	324.384646	301.764058	201.627734	55.025160	6.931289
s4	177.063238	115.712136	117.311732	48.259433	6.678027
s3	101.043268	-188.678898	-152.040280	-47.553816	-6.174550
s6	67.626692	86.749315	111.878956	44.213892	5.955597
age	-10.009866	1.308705	29.466112	19.812842	2.897090
sex	-239.815644	-207.192418	-83.154276	-0.918430	0.585254
s1	-792.175639	-83.466034	5.909614	19.924621	3.230957

규제 강도가
강할수록
회귀계수가
작아진다.

- 규제 선형 모델

131

- > 라쏘 회귀 : L1 규제

```
from sklearn.linear_model import Lasso
alphas = [0, 0.01, 0.1, 1, 10]
for alpha in alphas :
    lasso = Lasso(alpha = alpha)
    neg_mse_scores = cross_val_score(lasso, X, y,
                                     scoring="neg_mean_squared_error", cv = 5)
    avg_rmse = np.mean(np.sqrt(-1 * neg_mse_scores))
    print(f'alpha {alpha} 일 때 5 folds 의 평균 RMSE : {avg_rmse:.3f}')
```

Out :

```
alpha 0 일 때 5 folds 의 평균 RMSE : 54.692
alpha 0.01 일 때 5 folds 의 평균 RMSE : 54.756
alpha 0.1 일 때 5 folds 의 평균 RMSE : 54.843
alpha 1 일 때 5 folds 의 평균 RMSE : 62.009
alpha 10 일 때 5 folds 의 평균 RMSE : 77.264
```

- 규제 선형 모델

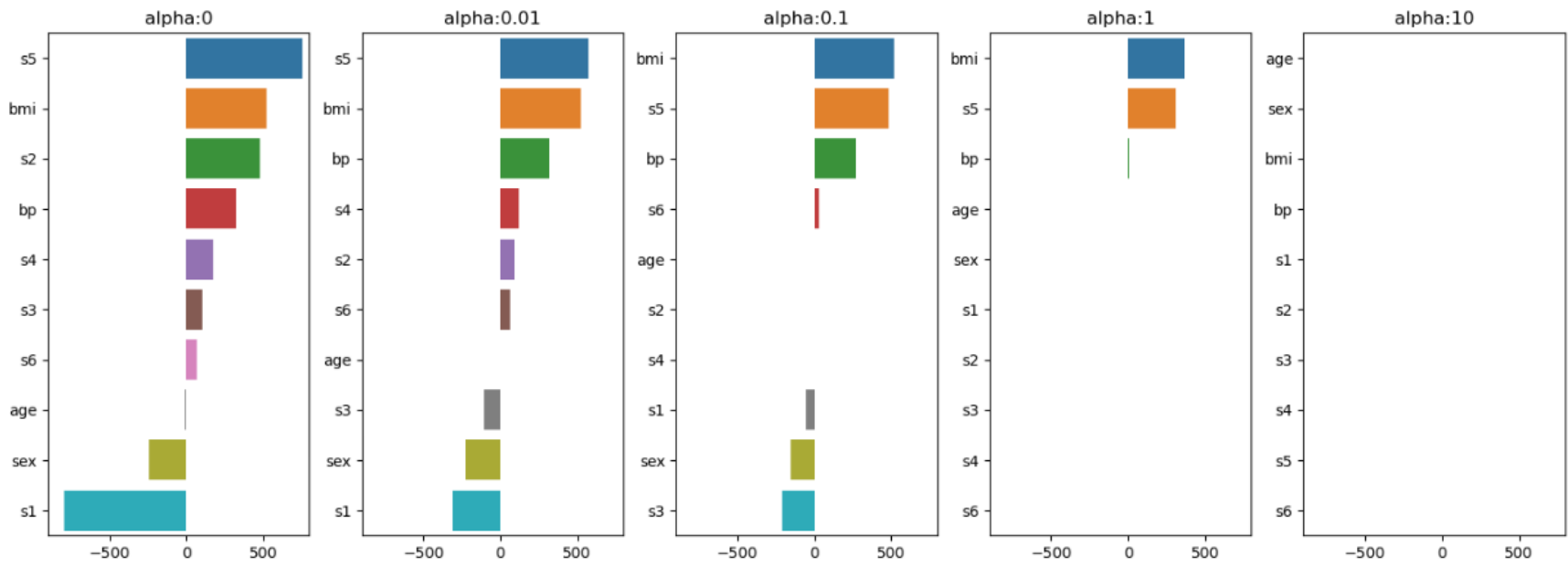
- > 라쏘 회귀 : L1 규제

```
fig, axs = plt.subplots(figsize = (18,6), nrows=1, ncols=5)
coeff_df = pd.DataFrame()
for pos, alpha in enumerate(alphas):
    lasso = Lasso(alpha = alpha)
    lasso.fit(X, y)
    coeff = pd.Series(data = lasso.coef_, index = X.columns)
    colname='alpha:'+ str(alpha)
    coeff_df[colname] = coeff
    coeff = coeff.sort_values(ascending = False)
    axs[pos].set_title(colname); axs[pos].set_xlim(-900, 800)
    sns.barplot(x = coeff.values, y=coeff.index, ax=axs[pos])
plt.show()
```

• 규제 선형 모델

> 라쏘 회귀 : L1 규제

Out :



• 규제 선형 모델

134

> 라쏘 회귀 : L1 규제

```
lasso_alphas = [0, 0.1, 1, 10, 100]
sort_column = 'alpha:' + str(lasso_alphas[0])
coeff_df.sort_values(by=sort_column, ascending=False)
```

Out :

	alpha:0	alpha:0.01	alpha:0.1	alpha:1	alpha:10
s5	751.273690	571.330356	483.912648	307.605418	0.0
bmi	519.845920	525.566130	517.186795	367.703860	0.0
s2	476.739000	89.324647	-0.000000	0.000000	0.0
bp	324.384645	316.168834	275.077235	6.298858	0.0
s4	177.063234	119.597616	0.000000	0.000000	0.0
s3	101.043256	-105.078369	-210.157991	-0.000000	-0.0
s6	67.626692	65.008383	33.673965	0.000000	0.0
age	-10.009866	-1.304662	-0.000000	0.000000	0.0
sex	-239.815644	-228.819129	-155.359976	-0.000000	0.0
s1	-792.175612	-307.016211	-52.539365	0.000000	0.0

L2 규제와 똑같이
규제 강도가 강할수록
회귀계수가 작아진다.

하지만 L1 규제는
강도가 강하면 0으로
수렴한다.

- 규제 선형 모델

135

- > 엘라스틱넷 회귀 : L1, L2 규제

```
from sklearn.linear_model import ElasticNet
alphas = [0, 0.01, 0.1, 1, 10]
for alpha in alphas :
    elastic = ElasticNet(alpha = alpha)
    neg_mse_scores = cross_val_score(elastic, X, y,
                                     scoring="neg_mean_squared_error", cv = 5)
    avg_rmse = np.mean(np.sqrt(-1 * neg_mse_scores))
    print(f'alpha {alpha} 일 때 5 folds 의 평균 RMSE : {avg_rmse:.3f}')
```

Out :

- alpha 0 일 때 5 folds 의 평균 RMSE : 54.692
- alpha 0.01 일 때 5 folds 의 평균 RMSE : 61.112
- alpha 0.1 일 때 5 folds 의 평균 RMSE : 73.203
- alpha 1 일 때 5 folds 의 평균 RMSE : 76.925
- alpha 10 일 때 5 folds 의 평균 RMSE : 77.264

- 규제 선형 모델

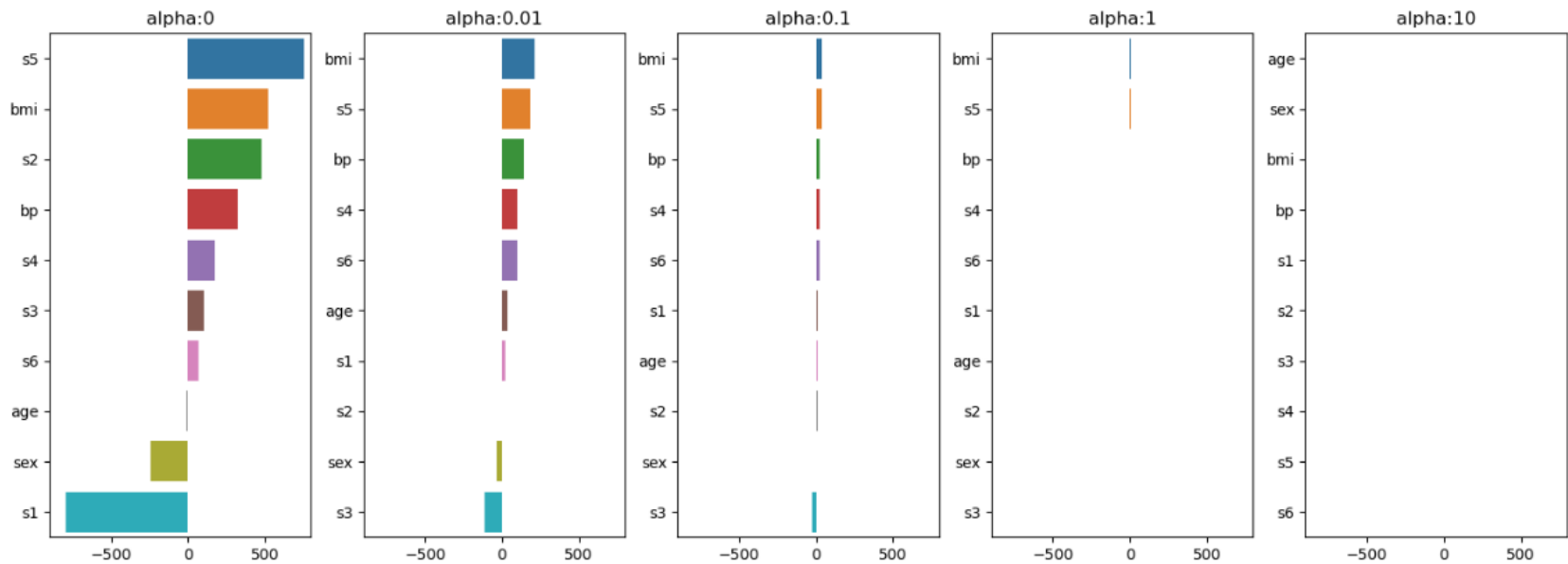
- > 엘라스틱넷 회귀 : L1, L2 규제

```
fig, axs = plt.subplots(figsize = (18,6), nrows=1, ncols=5)
coeff_df = pd.DataFrame()
for pos, alpha in enumerate(alphas):
    elastic = ElasticNet(alpha = alpha)
    elastic.fit(X, y)
    coeff = pd.Series(data = elastic.coef_, index = X.columns)
    colname='alpha:'+ str(alpha)
    coeff_df[colname] = coeff
    coeff = coeff.sort_values(ascending = False)
    axs[pos].set_title(colname); axs[pos].set_xlim(-900, 800)
    sns.barplot(x = coeff.values, y=coeff.index, ax=axs[pos])
plt.show()
```

- 규제 선형 모델

> 엘라스틱넷 회귀 : L1, L2 규제

Out :



• 규제 선형 모델

138

> 엘라스틱넷 회귀 : L1, L2 규제

```
alphas = [0, 0.1, 1, 10, 100]
sort_column = 'alpha:' + str(alphas[0])
coeff_df.sort_values(by=sort_column, ascending=False)
```

Out :

	alpha:0	alpha:0.01	alpha:0.1	alpha:1	alpha:10
s5	751.273690	185.325911	35.465700	3.105835	0.0
bmi	519.845920	211.024367	37.464655	3.259767	0.0
s2	476.739000	0.000000	8.355892	0.250935	0.0
bp	324.384645	144.559236	27.544765	2.204340	0.0
s4	177.063234	100.658917	25.505492	2.114454	0.0
s3	101.043256	-115.619973	-24.120809	-1.861363	-0.0
s6	67.626692	96.257335	22.894985	1.769851	0.0
age	-10.009866	33.147367	10.286332	0.359018	0.0
sex	-239.815644	-35.245354	0.285983	0.000000	0.0
s1	-792.175612	21.931722	11.108856	0.528646	0.0

마찬가지로 규제 강도가
클수록 회귀계수가
작아진다.

L1처럼 0으로
수렴하기도 하지만
L2처럼 천천히 값이
작아진다.

L1, L2 특징을 모두 표현

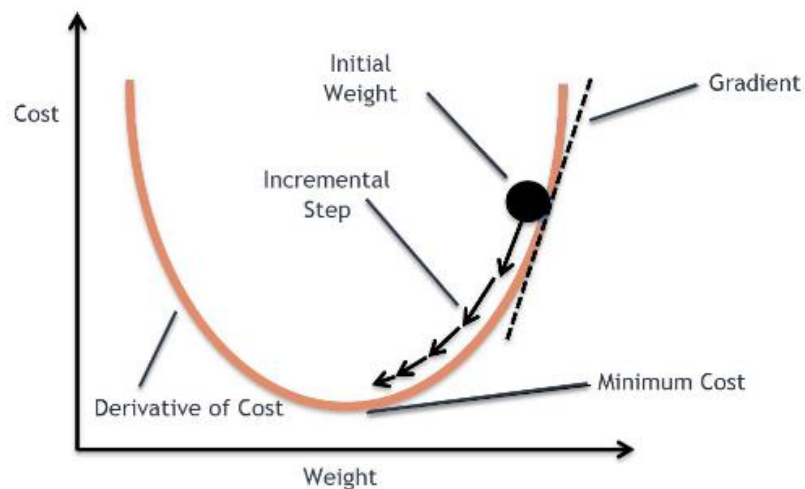
• 경사 하강법

> 비용(Cost)을 최소화 하는 방법

- 미분 값이 0이 되는 지점이 최소점

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$



출처: <https://towardsdatascience.com/using-machine-learning-to-predict-fitbit-sleep-scores-496a7d9ec48>

• 경사 하강법

140

> 확률적 경사 하강법(SGD) 실습

```
from sklearn.linear_model import SGDRegressor
alphas = [0, 0.01, 0.1, 1, 10]
for alpha in alphas :
    sgd_reg = SGDRegressor(alpha=alpha)
    neg_mse_scores = cross_val_score(sgd_reg, X, y,
                                     scoring="neg_mean_squared_error", cv = 5)
    avg_rmse = np.mean(np.sqrt(-1 * neg_mse_scores))
    print(f'alpha {alpha} 일 때 5 folds 의 평균 RMSE : {avg_rmse:.3f}')
```

Out : alpha 0 일 때 5 folds 의 평균 RMSE : 59.020
alpha 0.01 일 때 5 folds 의 평균 RMSE : 65.432
alpha 0.1 일 때 5 folds 의 평균 RMSE : 74.984
alpha 1 일 때 5 folds 의 평균 RMSE : 76.999
alpha 10 일 때 5 folds 의 평균 RMSE : 77.244

• 경사 하강법

141

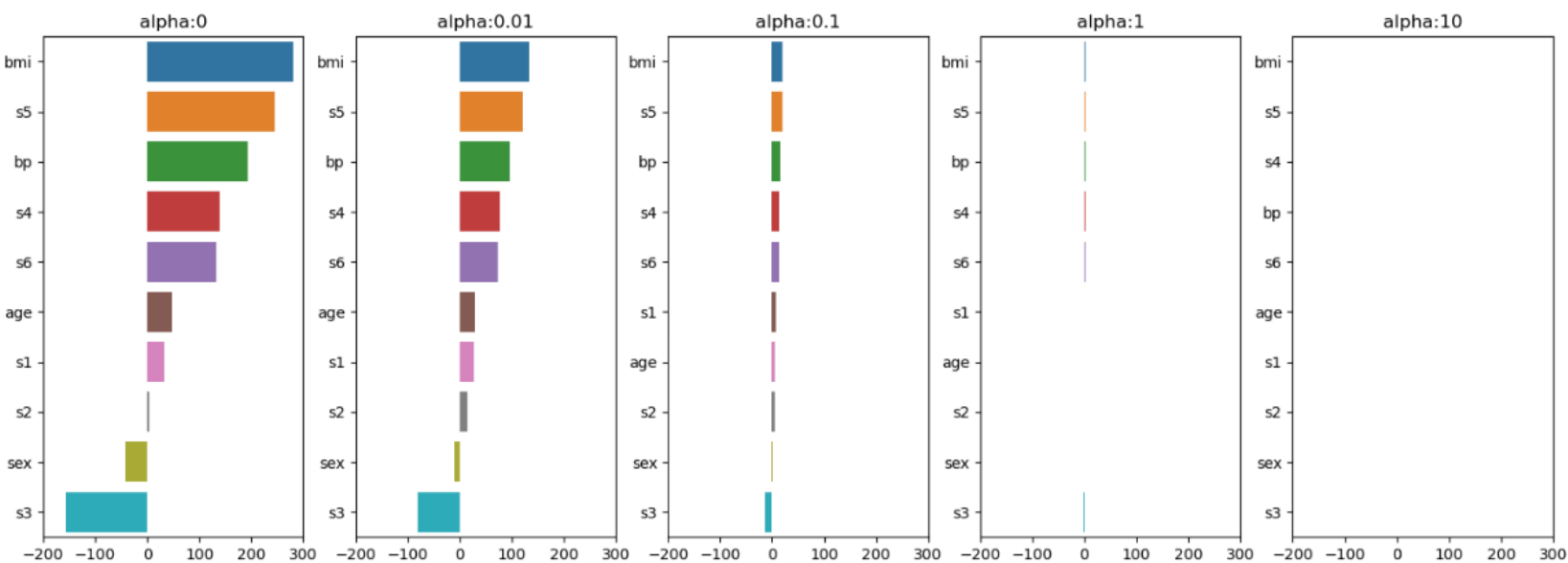
> 확률적 경사 하강법(SGD) 실습

```
fig, axs = plt.subplots(figsize = (18,6), nrows=1, ncols=5)
coeff_df = pd.DataFrame()
for pos, alpha in enumerate(alphas):
    sgd_reg = SGDRegressor(alpha = alpha)
    sgd_reg.fit(X, y)
    coeff = pd.Series(data = sgd_reg.coef_, index = X.columns)
    colname='alpha:'+ str(alpha)
    coeff_df[colname] = coeff
    coeff = coeff.sort_values(ascending = False)
    axs[pos].set_title(colname); axs[pos].set_xlim(-200, 300)
    sns.barplot(x = coeff.values, y=coeff.index, ax=axs[pos])
plt.show()
```

경사 하강법

> 확률적 경사 하강법(SGD) 실습

Out :



• 경사 하강법

143

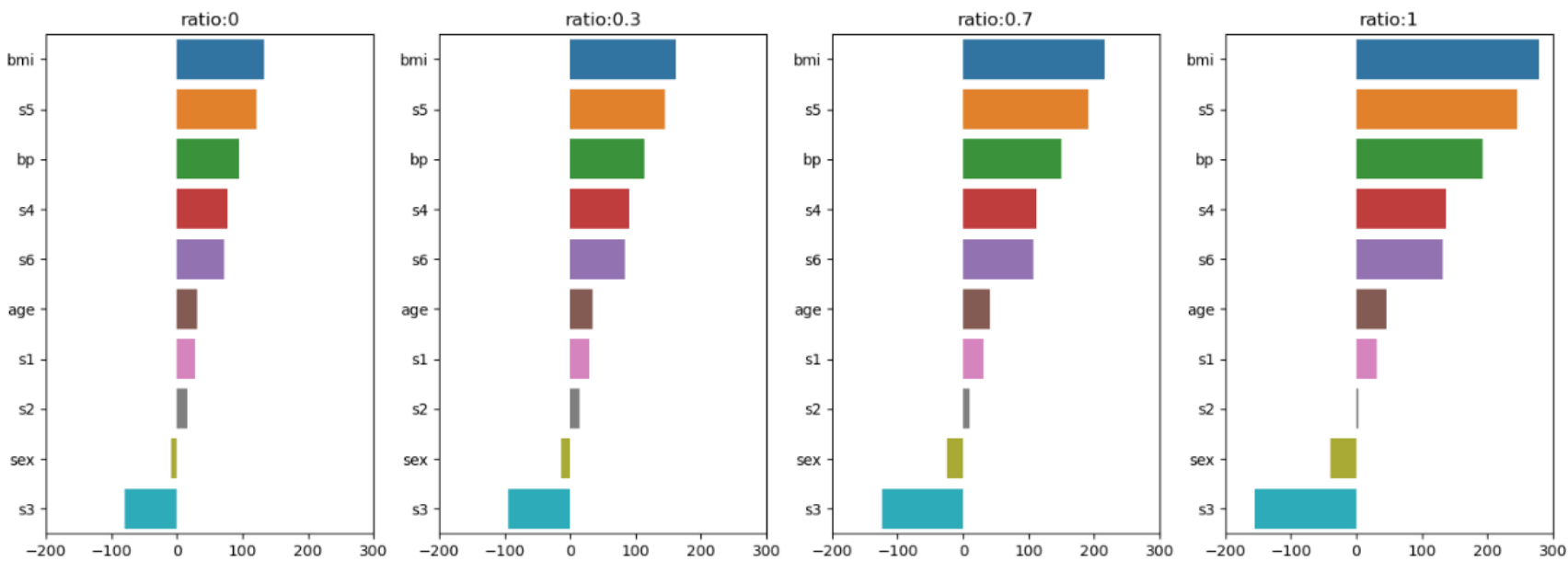
> 확률적 경사 하강법(SGD) 실습

```
fig, axs = plt.subplots(figsize = (18,6), nrow=1, ncol=4)
coeff_df = pd.DataFrame()
ratios = [0, 0.3, 0.7, 1]
for pos, ratio in enumerate(ratios):
    sgd_reg = SGDRegressor(alpha = 0.01, penalty='elasticnet', l1_ratio=ratio)
    sgd_reg.fit(X, y)
    coeff = pd.Series(data = sgd_reg.coef_, index = X.columns)
    colname='ratio:'+ str(ratio)
    coeff_df[colname] = coeff
    coeff = coeff.sort_values(ascending = False)
    axs[pos].set_title(colname); axs[pos].set_xlim(-200, 300)
    sns.barplot(x = coeff.values, y=coeff.index, ax=axs[pos])
plt.show()
```

- 경사 하강법

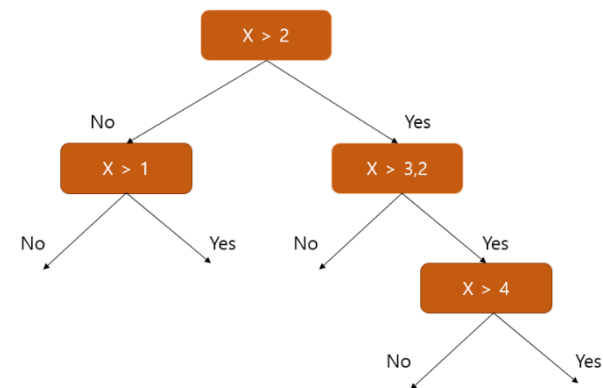
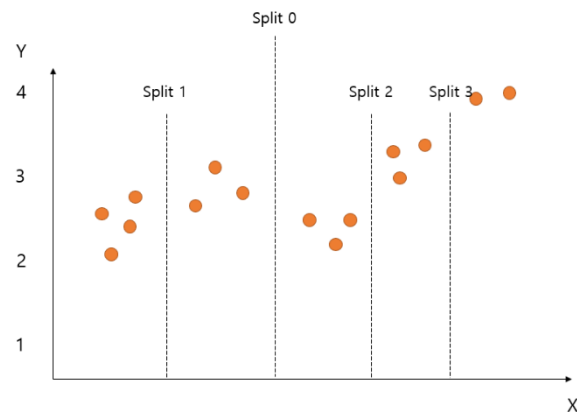
> 확률적 경사 하강법(SGD) 실습

Out :



• 회귀 트리

- > 회귀 트리란 회귀 함수(선형, 비선형)를 기반으로 하지 않고 트리 기반으로 회귀 예측을 하는 방식
- > 분류의 회귀와 크게 다르지 않지만 노드를 생성하는 방식이 다름
 - 클래스 대신 노드에 속한 데이터의 평균으로 값을 다룸
- > 분류에서 쓰이는 대부분의 트리 알고리즘들은 Classifier, Regression 2개로 구분되어 있다.



- 회귀 트리

- > 회귀 트리 실습

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import load_diabetes

diabetes = load_diabetes()

diabetes_df = pd.DataFrame(diabetes.data , columns = diabetes.feature_names)
diabetes_df['target'] = diabetes.target

y = diabetes_df.iloc[:, -1]
X = diabetes_df.iloc[:, :-1]
```

- 회귀 트리

147

- > 회귀 트리 실습

```
rf_reg = RandomForestRegressor(random_state=0, n_estimators=1000)
mse_scores = cross_val_score(rf_reg, X, y, scoring="neg_mean_squared_error", cv = 5)
rmse_scores = np.sqrt(-1 * mse_scores)
avg_rmse = np.mean(rmse_scores)
print(' 5 교차 검증의 개별 Negative MSE scores: ', np.round(mse_scores, 2))
print(' 5 교차 검증의 개별 RMSE scores : ', np.round(rmse_scores, 2))
print(f' 5 교차 검증의 평균 RMSE : {avg_rmse:.3f}')
```

Out : 5 교차 검증의 개별 Negative MSE scores: [-2994.11 -3112.26 -3547.44 -3272.69 -3709.92]
5 교차 검증의 개별 RMSE scores : [54.72 55.79 59.56 57.21 60.91]
5 교차 검증의 평균 RMSE : 57.637

- 회귀 트리

- > 회귀 트리 실습

```
def get_model_cv_prediction(model, X, y):  
    neg_mse_scores = cross_val_score(model, X, y,  
                                     scoring="neg_mean_squared_error", cv = 5)  
    rmse_scores = np.sqrt(-1 * neg_mse_scores)  
    avg_rmse = np.mean(rmse_scores)  
    print('##### ',model.__class__.__name__, ' #####')  
    print(' 5 교차 검증의 평균 RMSE : {0:.3f}'.format(avg_rmse))
```

- 회귀 트리

- > 회귀 트리 실습

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor

dt_reg = DecisionTreeRegressor(random_state=0, max_depth=4)
gb_reg = GradientBoostingRegressor(random_state=0, n_estimators=1000)
xgb_reg = XGBRegressor(random_state=0, n_estimators=1000)
```

- 회귀 트리

150

- > 회귀 트리 실습

```
models = [dt_reg, rf_reg, gb_reg, xgb_reg]
for model in models:
    get_model_cv_prediction(model, X, y)
```

Out : ##### DecisionTreeRegressor #####
5 교차 검증의 평균 RMSE : 64.320
RandomForestRegressor #####
5 교차 검증의 평균 RMSE : 57.637
GradientBoostingRegressor #####
5 교차 검증의 평균 RMSE : 65.495
XGBRegressor #####
5 교차 검증의 평균 RMSE : 63.035

- 회귀 트리

151

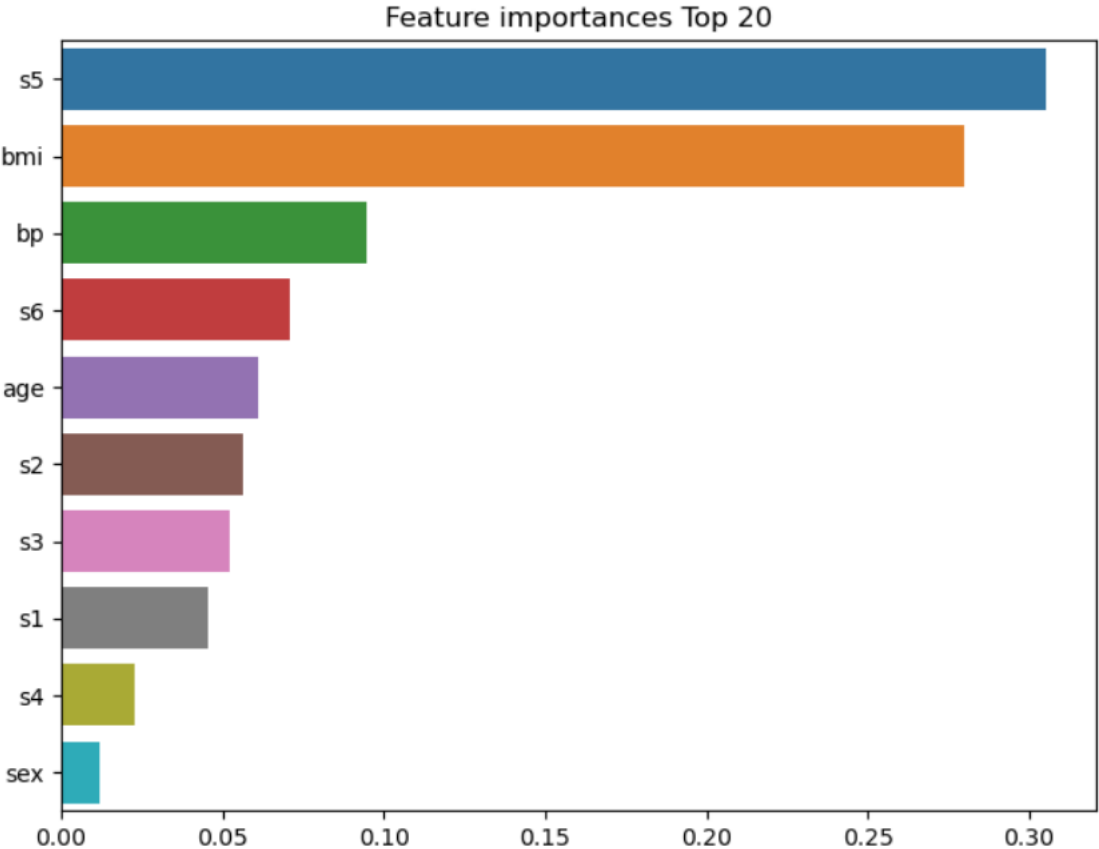
- > 회귀 트리 동작 비교

```
rf_reg.fit(X, y)
ftr_importances_values = rf_reg.feature_importances_
ftr_importances = pd.Series(ftr_importances_values, index=X.columns)
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20, y=ftr_top20.index)
plt.show()
```

- 회귀 트리

> 회귀 트리 동작 비교

Out :



- 회귀 트리

153

- > 회귀 트리 동작 비교

```
import matplotlib.pyplot as plt

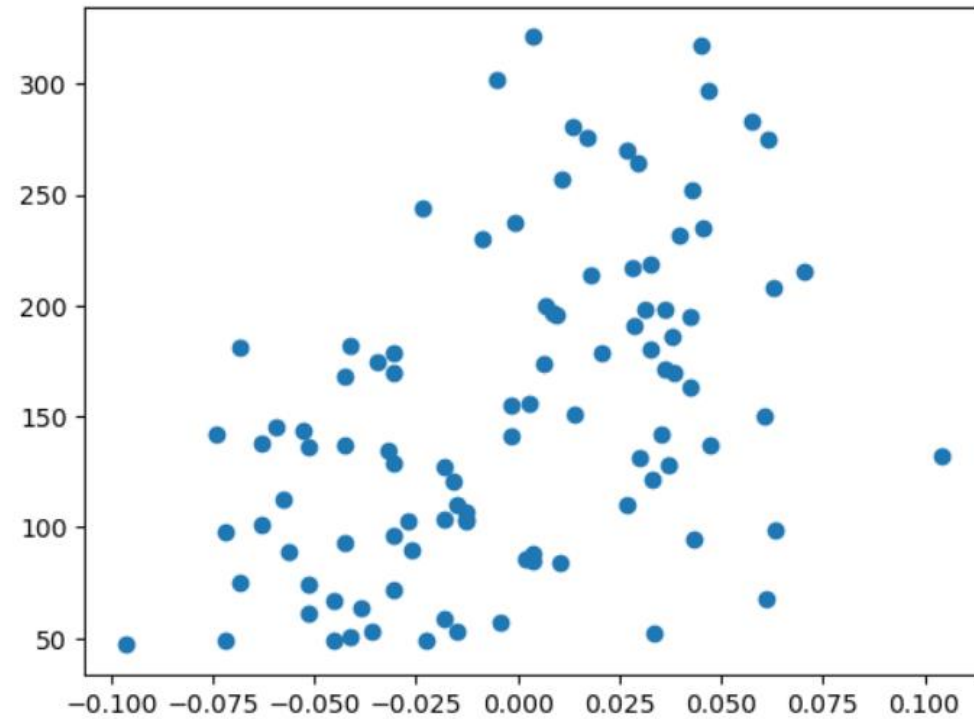
sample_df = diabetes_df[['s5','target']]
sample_df = sample_df.sample(n=100,random_state=0)
print(sample_df.shape)
plt.figure()
plt.scatter(sample_df['s5'] , sample_df['target'])
```

Out : (100, 2)

- 회귀 트리

- > 회귀 트리 동작 비교

Out :



- 회귀 트리

- > 회귀 트리 동작 비교

```
import numpy as np
from sklearn.linear_model import LinearRegression

lr_reg = LinearRegression()
dt_reg = DecisionTreeRegressor()

X_test = np.linspace(-0.1, 0.1, 50).reshape(-1,1)
X_feature = sample_df['s5'].values.reshape(-1,1)
y_target = sample_df['target'].values.reshape(-1,1)

lr_reg.fit(X_feature, y_target)
dt_reg.fit(X_feature, y_target)
```

- 회귀 트리

- > 회귀 트리 동작 비교

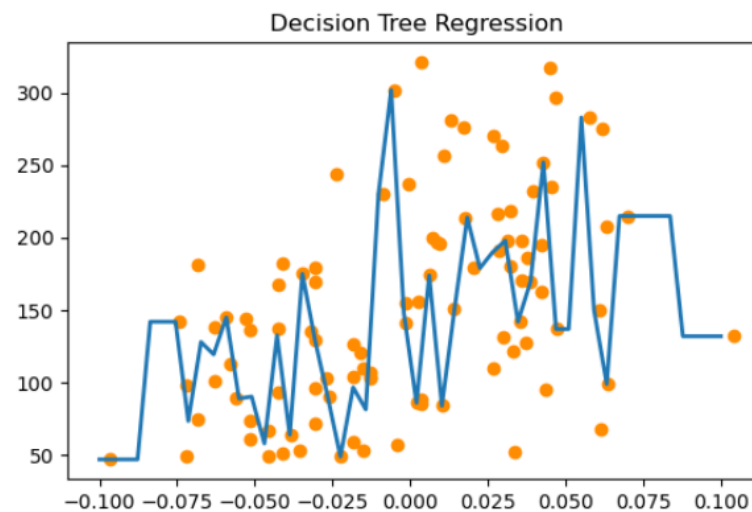
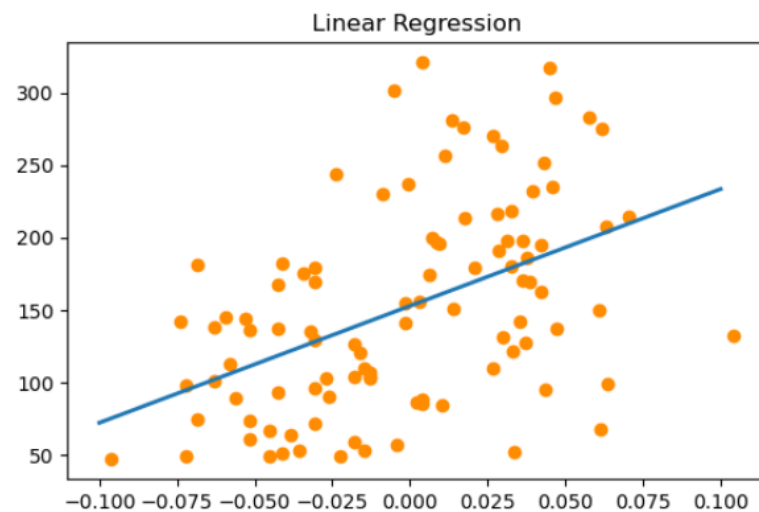
```
pred_lr = lr_reg.predict(X_test)
pred_dt = dt_reg.predict(X_test)

fig , (ax1, ax2) = plt.subplots(figsize=(14,4), ncols=2)
ax1.set_title('Linear Regression')
ax1.scatter(sample_df['s5'], sample_df['target'], c="darkorange")
ax1.plot(X_test, pred_lr, linewidth=2 )
ax2.set_title('Decision Tree Regression')
ax2.scatter(sample_df['s5'], sample_df['target'], c="darkorange")
ax2.plot(X_test, pred_dt, linewidth=2 )
```

- 회귀 트리

- > 회귀 트리 동작 비교

Out :



- 회귀 트리

- > 회귀 트리 동작 비교

```
dt_reg1 = DecisionTreeRegressor(max_depth=3)
dt_reg1.fit(X_feature, y_target)
pred_dt1 = dt_reg1.predict(X_test)
dt_reg2 = DecisionTreeRegressor(max_depth=5)
dt_reg2.fit(X_feature, y_target)
pred_dt2 = dt_reg2.predict(X_test)
```

- 회귀 트리

- > 회귀 트리 동작 비교

```
fig , (ax1, ax2) = plt.subplots(figsize=(14,4), ncols=2)
ax1.set_title('Decision Tree Regression\nMax_depth = 3')
ax1.scatter(sample_df['s5'], sample_df['target'], c="darkorange")
ax1.plot(X_test, pred_dt1, linewidth=2 )
ax2.set_title('Decision Tree Regression\nMax_depth = 5')
ax2.scatter(sample_df['s5'], sample_df['target'], c="darkorange")
ax2.plot(X_test, pred_dt2, linewidth=2 )
```

- 회귀 트리

- > 회귀 트리 동작 비교

Out :

